

République Française



Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université Sorbonne Paris Nord
École d'Ingénieurs Sup'Galilée



Par

Chaima JOUINI et Tedj el moulk SINACER

Conception et Développement d'une Application de Navigation Indoor Basée sur des Capteurs Inertiels de Smartphone

Encadrant :

Christophe Daussy

Table des matières

Introduction générale	1
1 Contexte général	2
1.1 Problématique	3
1.2 Étude et critique de l'existant	4
1.2.1 Solutions existantes	4
1.2.2 Critiques de l'existant	5
1.3 Cahier des charges	6
1.3.1 Méthodologie de travail	7
2 Principes physiques, traitement inertiel et planification de trajectoire	9
2.1 Détection des pas	10
2.1.1 Traitement du signal et choix du filtrage	10
2.1.2 Filtrage Biquad Passe-Bande	11
2.1.3 Filtrage du gyroscope	13
2.1.4 Étalonnage et calibration utilisateur	14
2.2 Calcul de l'orientation	17
2.2.1 Principe physique	17
2.2.2 Capteurs utilisés	17
2.2.3 Conversion en angles d'Euler	17
2.2.4 Normalisation angulaire	18
2.2.5 Filtrage circulaire	18
2.2.6 Déroulement continu (Unwrap)	19
2.2.7 Passage au repère carte	20
2.2.8 Rôle dans la navigation	20
2.3 Planification de trajectoire par algorithme A*	20
2.3.1 Contexte et motivation	20
2.3.2 Représentation de la carte	21
2.3.3 Principe de l'algorithme A*	21
2.3.4 Voisinage et coûts	21
2.3.5 Recherche et arrêt	22
2.3.6 Reconstruction et lissage du chemin	22

2.3.7	Intégration avec la navigation inertielle	22
2.3.8	Gestion et validation des virages	22
3	Conception et architecture de la solution proposée	30
3.1	Vue d'ensemble de la solution de navigation indoor	31
3.2	Architecture globale du système	32
3.2.1	Architecture logicielle générale	32
3.2.2	Découpage fonctionnel de l'application Android	33
3.3	Logique de navigation et enchaînement des traitements	34
3.3.1	Initialisation et prérequis de navigation	34
3.3.2	Définition du trajet et préparation de la navigation	38
3.3.3	Suivi de la progression et mise à jour de la position de l'utilisateur sur le trajet	39
3.4	Gestion intelligente des virages	40
3.4.1	Détection des virages sur le trajet	40
3.4.2	Mécanisme de verrouillage de virage (Turn Lock)	41
3.4.3	Stratégies de sécurité et cas particuliers	43
3.5	Interaction utilisateur et retour visuel	44
3.6	Communication avec MATLAB et outils d'analyse	44
4	Réalisation	46
4.1	Présentation de l'application finale	47
4.2	Mise en œuvre expérimentale et conditions de test	50
4.2.1	Environnement de test	50
4.2.2	Scénarios de test	50
4.3	Résultats expérimentaux obtenus	51
4.3.1	Environnement et protocole de test	51
4.3.2	Performance de la détection des pas	51
4.3.3	Précision de l'estimation de distance	51
4.3.4	Stabilité de l'orientation	51
4.3.5	Robustesse globale du système	52
4.4	Analyse et discussion des résultats	52
4.5	Retours d'expérience des utilisateurs	53
4.6	Perspectives et améliorations possibles	53
	Conclusion générale	55

Table des figures

2.1	en haut, visualisation du signal brut de l'accélération et du signal filtré, ainsi que de la norme de l'accélération en bas.	13
3.1	Vue générale du fonctionnement de la solution proposée	32
3.2	Architecture logicielle de l'application de navigation indoor	33
3.3	Architecture fonctionnelle de l'application de navigation indoor	34
3.4	Configuration initiale du module de navigation	35
3.5	Chargement des paramètres utilisateur au démarrage	35
3.6	Chargement du plan du bâtiment et préparation des zones navigables	36
3.7	Implémentation des fonctions de détection et validation des zones accessibles	36
3.8	Chargement des paramètres utilisateur	37
3.9	Initialisation des capteurs IMU	37
3.10	Vérification des conditions préalables au lancement de la navigation	37
3.11	Configuration des interactions utilisateur et des commandes de navigation	38
3.12	Implémentation du calcul du chemin et vérification de validité	38
3.13	Mise à jour de la position de l'utilisateur après détection d'un pas	39
3.14	Projection de la position estimée sur le chemin calculé	40
3.15	Détection des changements de direction le long du trajet	41
3.16	Implémentation du mécanisme de verrouillage de virage (Turn Lock)	41
3.17	Activation et initialisation du mécanisme de Turn Lock	42
3.18	Gestion et contrôle du verrouillage pendant la phase de virage	42
3.19	Validation du virage et reprise de la progression	43
3.20	Implémentation des mécanismes de sécurité et gestion des cas particuliers	43
4.1	Phase d'étalonnage personnalisé de la marche (5 m)	48
4.2	Processus de définition du parcours sur le plan intérieur	49
4.3	Fonctionnalités interactives de contrôle du parcours	49

Introduction générale

Dans le cadre du projet de fin d'études de la spécialité Instrumentation et Systèmes Embarqués, nous avons développé une application de navigation intérieure reposant exclusivement sur les capteurs embarqués d'un smartphone.

L'objectif principal de ce travail est de proposer une solution de navigation indoor capable de fonctionner sans infrastructure externe, en exploitant uniquement les données issues de l'unité de mesure inertielle (IMU) intégrée au téléphone. Cette approche répond aux limitations des systèmes GNSS en environnement intérieur, où les signaux satellitaires deviennent inexploitable.

Afin d'aboutir à un produit final cohérent et fonctionnel, nous avons dans un premier temps réalisé une étude bibliographique approfondie portant sur les méthodes de navigation piétonne, le traitement du signal inertiel et les algorithmes de planification de trajectoire. Cette phase de recherche nous a permis d'enrichir nos connaissances théoriques et de définir un cahier des charges précis, adapté aux contraintes techniques d'un système embarqué temps réel.

Le développement de la solution a ensuite été mené de manière progressive et structurée, en intégrant les différentes briques fonctionnelles : détection de pas, estimation d'orientation, calibration personnalisée, planification par algorithme A*, ainsi que validation physique des virages.

L'application a été testée par plusieurs étudiants afin d'évaluer ses performances en conditions réelles d'utilisation. Ces expérimentations ont permis d'analyser la robustesse du système, d'identifier ses limites et d'apporter des améliorations ciblées.

Les chapitres suivants détaillent l'ensemble des étapes du projet, depuis l'étude théorique jusqu'à l'implémentation pratique. Ils présentent les traitements appliqués aux signaux, les choix algorithmiques retenus, les difficultés rencontrées, ainsi que les solutions mises en œuvre pour garantir la cohérence physique et la stabilité du système.

Ce travail illustre ainsi la conception complète d'un système embarqué combinant modélisation physique, traitement du signal et planification algorithmique, appliqué au domaine de la navigation intérieure.

CONTEXTE GÉNÉRAL

Plan

1	Problématique	3
2	Étude et critique de l'existant	4
3	Cahier des charges	6

Introduction

Ce chapitre présente le contexte général du projet réalisé dans le cadre du projet de fin d'études. Il débute par une mise en contexte de la problématique à laquelle ce projet cherche à répondre, à savoir la navigation en environnement intérieur (navigation indoor). Une étude des solutions existantes est ensuite présentée afin d'identifier leurs limites et de justifier le choix de l'approche adoptée. Enfin, les objectifs généraux du projet ainsi que les principales contraintes sont exposés, avant de conclure par une présentation de l'organisation globale du travail mise en place pour la réalisation de ce projet

1.1 Problématique

Le système de positionnement par satellite (GPS) est aujourd'hui largement utilisé pour la navigation en extérieur grâce à sa précision et à sa disponibilité. Cependant, son utilisation devient très limitée en environnement intérieur. En effet, les signaux GPS sont fortement atténués par les murs, les plafonds et les structures des bâtiments, ce qui entraîne une perte de précision, voire une absence totale de signal. Dans des lieux fermés tels que les universités, les hôpitaux, les centres commerciaux ou les parkings souterrains, le GPS ne permet donc pas d'assurer une localisation fiable. Cette limitation constitue un obstacle majeur pour les applications de navigation indoor.

Face aux limites du GPS en milieu intérieur, il devient nécessaire de développer des solutions alternatives capables d'assurer une localisation et une navigation fiables à l'intérieur des bâtiments. Une telle solution doit permettre de guider un utilisateur entre un point de départ et un point d'arrivée, tout en prenant en compte ses déplacements réels. Pour cela, il est possible d'exploiter les capteurs et les technologies déjà intégrés dans les smartphones, tels que les capteurs inertiels, ainsi que des moyens de communication sans fil comme le Wi-Fi ou le Bluetooth. Ces technologies offrent l'avantage d'être largement disponibles et de ne pas nécessiter, dans certains cas, l'installation d'infrastructures lourdes. L'objectif est donc de proposer une solution de navigation indoor capable de fonctionner de manière autonome et continue, tout en s'adaptant aux contraintes des environnements intérieurs.

La mise en place d'un système de navigation indoor doit respecter plusieurs contraintes réelles. Tout d'abord, le coût doit rester faible afin de garantir une solution accessible et facilement déployable, sans nécessiter l'installation d'équipements spécifiques. Ensuite, la consommation énergétique est un critère important, en particulier pour les applications mobiles, afin de préserver l'autonomie du smartphone. Enfin, la précision du système doit être suffisante pour assurer une navigation efficace, tout en tenant compte des erreurs cumulées liées aux capteurs inertiels. Le défi consiste donc à trouver un compromis entre précision, coût et consommation énergétique.

1.2 Étude et critique de l'existant

Afin de mieux comprendre la problématique de la navigation en environnement intérieur et de situer le travail réalisé, il est nécessaire d'analyser les solutions déjà existantes dans ce domaine. Cette étude permet d'identifier les approches les plus utilisées ainsi que leurs limites, et de justifier le choix de la solution proposée dans ce projet.

1.2.1 Solutions existantes

Plusieurs solutions ont été développées pour assurer la localisation et la navigation à l'intérieur des bâtiments. Ces solutions peuvent être classées en deux grandes catégories :

- Les méthodes nécessitant une infrastructure installée dans le bâtiment (Wi-Fi, Bluetooth, UWB, etc.).
- Les méthodes autonomes basées sur des capteurs inertiels (IMU – *Inertial Measurement Unit*).

Localisation basée sur le Wi-Fi

Les systèmes basés sur le Wi-Fi utilisent l'intensité du signal des points d'accès afin d'estimer la position de l'utilisateur. Deux approches principales sont utilisées :

- **La trilatération**, qui estime la distance entre l'utilisateur et plusieurs bornes Wi-Fi à partir de la puissance du signal reçu (Received Signal Strength Indicator – RSSI).
- **Le fingerprinting**, qui consiste à effectuer au préalable une cartographie du bâtiment en enregistrant les caractéristiques des signaux Wi-Fi à différents emplacements. Lors de l'utilisation, le système compare les signaux mesurés en temps réel à cette base de données afin d'identifier la position la plus probable.

La précision des systèmes Wi-Fi varie généralement entre 2 et 5 mètres, selon la densité des bornes et les conditions environnementales.

Cette approche est largement répandue car elle exploite une infrastructure souvent déjà disponible.

Localisation basée sur le Bluetooth (BLE)

D'autres solutions reposent sur la technologie Bluetooth Low Energy (BLE), notamment à travers l'utilisation de balises installées dans le bâtiment. Le système estime la position de l'utilisateur en fonction de la puissance du signal reçu depuis ces balises.

Cette méthode offre une précision généralement comprise entre 1 et 3 mètres. Elle est largement utilisée dans les centres commerciaux, musées ou aéroports. Cependant, elle nécessite l'installation et la maintenance régulière des balises.

Ultra Wide Band (UWB)

Certaines technologies plus récentes utilisent l’Ultra Wide Band (UWB), qui permet une localisation très précise (de l’ordre de 10 à 30 centimètres) grâce à la mesure du temps de propagation des signaux radio.

Cependant, cette solution nécessite l’installation d’équipements compatibles dans le bâtiment ainsi que des dispositifs capables de communiquer en UWB, ce qui peut engendrer un coût de déploiement élevé.

Méthodes basées sur des capteurs inertiels (IMU)

La navigation indoor peut également s’appuyer sur la fusion de capteurs inertiels, notamment l’accéléromètre, le gyroscope et le magnétomètre.

Cette approche repose généralement sur la méthode du Pedestrian Dead Reckoning (PDR), qui consiste à :

- détecter les pas
- estimer la longueur du pas
- déterminer la direction du déplacement
- mettre à jour progressivement la position de l’utilisateur

La position est obtenue par intégration successive des déplacements mesurés. L’avantage principal de cette méthode est qu’elle ne nécessite aucune infrastructure externe.

Vision artificielle et SLAM

Enfin, certaines méthodes utilisent la vision artificielle et des techniques d’intelligence artificielle, telles que le SLAM (*Simultaneous Localization and Mapping*), pour analyser l’environnement intérieur et estimer la position de l’utilisateur en temps réel.

1.2.2 Critiques de l’existant

Malgré leur efficacité dans certains contextes, ces solutions présentent plusieurs limites.

Les systèmes basés sur le Wi-Fi et le Bluetooth dépendent fortement de l’infrastructure existante et peuvent être sensibles aux interférences, aux obstacles physiques ainsi qu’aux variations de signal. De plus, les méthodes de fingerprinting nécessitent une phase initiale de cartographie longue et coûteuse, ainsi qu’une mise à jour régulière en cas de modification de l’environnement intérieur.

Les solutions utilisant l’UWB offrent une excellente précision, mais leur déploiement implique un coût supplémentaire lié à l’installation de matériel spécifique.

Les méthodes inertielle basées sur IMU sont autonomes et peu coûteuses, mais souffrent d'une dérive cumulative inhérente à l'intégration des mesures. En effet, de petites erreurs dans la mesure de l'accélération ou de l'orientation s'accumulent progressivement, entraînant une augmentation de l'erreur de position au fil du temps. Sans mécanisme de correction externe, la précision diminue progressivement.

Ainsi, aucune des solutions existantes ne permet de répondre pleinement aux exigences simultanées de précision, de coût et de robustesse imposées par la navigation indoor, ce qui met en évidence la nécessité de proposer une solution adaptée et optimisée.

1.3 Cahier des charges

La solution de navigation indoor développée dans le cadre de ce projet vise à fournir un système fiable et autonome permettant de guider un utilisateur à l'intérieur d'un bâtiment fermé, sans dépendre du système GPS ni d'une infrastructure externe coûteuse.

Le système devra, dans un premier temps, permettre à l'utilisateur de définir simplement un point de départ ainsi qu'un point d'arrivée sur un plan intérieur du bâtiment. À partir de ces informations, l'application devra calculer un itinéraire et accompagner l'utilisateur tout au long de son déplacement, en lui fournissant des indications de navigation claires et compréhensibles.

La solution reposera principalement sur l'exploitation des capteurs embarqués dans le smartphone, notamment l'accéléromètre, le gyroscope et le magnétomètre. Ces capteurs permettront d'estimer les déplacements de l'utilisateur ainsi que son orientation, selon une approche de navigation inertielle. Ce choix vise à garantir une autonomie totale du système, sans nécessité d'installer des équipements supplémentaires tels que des bornes Wi-Fi, des balises Bluetooth ou des capteurs dédiés.

Le système devra fonctionner en temps réel afin d'assurer une navigation fluide et réactive. Une interface mobile intuitive sera développée pour faciliter l'interaction avec l'utilisateur, notamment pour la sélection des points de départ et d'arrivée, l'affichage du plan du bâtiment et la visualisation de l'itinéraire suivi. L'ergonomie de l'application devra être adaptée à un usage quotidien et accessible à tout type d'utilisateur.

En termes de performances, la précision de la navigation devra être suffisante pour un usage pratique en environnement intérieur. Le système devra limiter autant que possible l'accumulation des erreurs inhérentes aux capteurs inertiels, notamment par l'utilisation de techniques de filtrage et de correction adaptées. Un compromis devra être trouvé entre précision, robustesse et simplicité de mise en œuvre.

Enfin, la solution devra présenter une consommation énergétique réduite afin de préserver l'autonomie du smartphone lors d'une utilisation prolongée. Elle devra également être facilement

déployable et adaptable à différents types d'environnements intérieurs, tels que les universités, les bâtiments administratifs ou les espaces publics, sans nécessiter de modifications matérielles importantes.

1.3.1 Méthodologie de travail

La réalisation de ce projet a reposé sur une méthodologie de travail collaborative et progressive, menée par l'équipe composée de Chaima Jouini et Tedj el Moulk Sinacer. Cette organisation a permis d'assurer une bonne répartition des tâches, un suivi continu de l'avancement et une amélioration progressive de la solution développée.

La première étape du travail a consisté en une phase de recherche approfondie visant à comprendre les différentes solutions existantes dans le domaine de la navigation en environnement intérieur. Cette phase a été réalisée de manière individuelle, chaque membre de l'équipe étudiant séparément les approches proposées dans la littérature scientifique et les projets existants. L'objectif de cette démarche était d'identifier les solutions les plus pertinentes, leurs avantages ainsi que leurs limites, afin d'orienter le choix technologique du projet.

À l'issue de cette phase de recherche, des réunions régulières ont été organisées afin de partager les informations collectées, confronter les points de vue et discuter des différentes possibilités techniques. Ces échanges ont permis d'aboutir à un choix commun, à savoir une solution de navigation indoor basée principalement sur l'exploitation des capteurs intégrés aux smartphones. Ce choix a été motivé par la volonté de proposer une solution accessible au plus grand nombre, ne nécessitant aucun déploiement d'infrastructure supplémentaire, tout en restant adaptée aux contraintes réelles des environnements intérieurs.

Une fois cette orientation définie, l'équipe a entamé la phase de développement de l'application mobile. Cette phase a été réalisée de manière itérative, en commençant par la mise en place des fonctionnalités de base, notamment la récupération des données issues des capteurs inertiels du smartphone et leur traitement. L'envoi des données vers l'environnement MATLAB via le protocole UDP a également été implémenté, ce qui a permis de visualiser, analyser et valider les données en temps réel. Cette approche a facilité le débogage et l'amélioration progressive des algorithmes développés.

Tout au long du développement, une gestion de version via GitHub a été mise en place. Chaque mise à jour du code était régulièrement poussée vers le dépôt distant, permettant aux deux membres de l'équipe de rester synchronisés et de travailler sur une base de code commune. Cette pratique a favorisé un travail collaboratif efficace, tout en assurant la traçabilité des modifications et la possibilité de revenir à des versions antérieures si nécessaire.

Par ailleurs, des réunions de suivi ont été organisées après chaque étape importante afin d'évaluer l'état d'avancement du projet, de planifier les prochaines tâches et de répartir les objectifs à court terme.

Dans certains cas, les deux membres ont également travaillé en parallèle sur un même problème en utilisant des approches différentes, ce qui a permis de comparer les résultats obtenus, d'identifier la solution la plus efficace et d'améliorer la robustesse globale du système.

Ainsi, cette méthodologie de travail basée sur la recherche, la collaboration, l'itération et le suivi continu a permis de mener à bien le projet dans de bonnes conditions, tout en garantissant une progression cohérente et maîtrisée vers les objectifs fixés.

Conclusion

Ce chapitre a permis d'introduire le cadre général du projet en mettant en évidence la problématique de la navigation en environnement intérieur et les limites des solutions actuellement disponibles. L'étude de l'existant a montré que, malgré les avancées technologiques, aucune approche ne répond pleinement aux exigences de précision, de coût et de simplicité d'utilisation en milieu fermé. À partir de ce constat, le cahier des charges a été défini afin de préciser les objectifs et les contraintes de la solution proposée. Enfin, la méthodologie de travail adoptée a été présentée, mettant en avant l'organisation du projet et la collaboration entre les membres de l'équipe. Ce chapitre constitue ainsi une base solide pour aborder, dans le chapitre suivant, la conception et l'architecture détaillée du système de navigation indoor développé.

PRINCIPES PHYSIQUES, TRAITEMENT INERTIEL ET PLANIFICATION DE TRAJECTOIRE

Plan

1	Detection des pas	10
2	Calcul de l'orientation	17
3	Planification de trajectoire par algorithme A*	20

Introduction

Dans ce chapitre, nous présentons les fondements physiques et les méthodes de traitement du signal sur lesquels repose l'application. Nous détaillons les différents mécanismes mis en œuvre pour détecter les pas à partir des données issues des capteurs inertiels, estimer l'orientation de l'utilisateur, calculer le trajet optimal, guider l'utilisateur le long du chemin, détecter et valider les virages en temps réel. L'objectif est d'expliquer le fonctionnement global du système, depuis l'acquisition des données des capteurs jusqu'à la prise de décision en navigation. Nous décrivons ainsi comment l'application combine modélisation physique, traitement du signal et planification de trajectoire afin d'aboutir à un système de navigation intérieure temps réel, robuste et cohérent.

2.1 Détection des pas

Après une analyse approfondie des différentes approches de navigation intérieure proposées dans la littérature scientifique, nous avons retenu la méthode PDR (Pedestrian Dead Reckoning) comme solution principale. Ce choix repose essentiellement sur sa robustesse en environnement indoor, où les systèmes GNSS deviennent inexploitable, ainsi que sur son faible coût matériel, puisqu'elle exploite uniquement les capteurs inertiels déjà intégrés dans les smartphones modernes.

Les smartphones embarquent une unité de mesure inertielle composée d'un accéléromètre, d'un gyroscope et d'un magnétomètre. Ces capteurs MEMS permettent de mesurer respectivement l'accélération linéaire, la vitesse angulaire et le champ magnétique terrestre. Grâce aux interfaces logicielles du système Android, il est possible d'accéder en temps réel aux données brutes de ces capteurs via une application mobile.

Dans une première phase expérimentale, nous avons utilisé MATLAB afin d'observer les signaux bruts issus des différents capteurs et de comprendre leur comportement physique. Cette étape a permis d'analyser la structure des données, d'identifier les composantes pertinentes pour la détection des pas et l'estimation de l'orientation, et de poser les bases du traitement du signal nécessaire à la navigation.

La logique générale du système repose ainsi sur l'exploitation des mesures inertielle pour estimer le déplacement local de l'utilisateur, tout en appliquant des techniques de traitement adaptées afin d'extraire l'information utile contenue dans les signaux bruts.

2.1.1 Traitement du signal et choix du filtrage

L'analyse des données de l'accéléromètre montre que le signal mesuré est constitué de plusieurs composantes superposées. On peut l'exprimer sous la forme :

$$a_{\text{mes}}(t) = a_{\text{mouvement}}(t) + g \quad (2.1)$$

où $g \approx 9.81 \text{ m/s}^2$ représente l'accélération gravitationnelle, et $a_{\text{mouvement}}(t)$ correspond aux accélérations dynamiques liées aux déplacements de l'utilisateur.

La présence de la gravité, combinée aux bruits haute fréquence et aux micro-perturbations, rend indispensable l'utilisation d'un filtrage adapté pour isoler la dynamique propre à la marche. D'après les travaux en biomécanique, la fréquence caractéristique de la marche humaine se situe approximativement entre 0.7 Hz et 3 Hz, plage correspondant à la cadence physiologique normale d'un piéton.

Sur cette base, nous avons opté pour un filtrage passe-bande dans cet intervalle fréquentiel afin de supprimer la composante quasi statique liée à la gravité, d'atténuer les perturbations rapides et de conserver uniquement les oscillations associées aux pas.

Plusieurs méthodes de filtrage ont été étudiées, notamment le filtre de Kalman, largement utilisé dans les systèmes d'estimation d'état. Toutefois, ce type de filtre nécessite des opérations matricielles répétées ainsi qu'une modélisation dynamique plus complexe, ce qui augmente la charge computationnelle. Dans le contexte d'une application mobile fonctionnant en temps réel, une solution plus légère était préférable.

Nous avons ainsi retenu un filtre biquad passe-bande, qui présente un bon compromis entre efficacité fréquentielle, stabilité numérique et faible coût de calcul. Ce choix permet d'obtenir un signal exploitable pour la détection des pas tout en respectant les contraintes de performance d'un smartphone.

2.1.2 Filtrage Biquad Passe-Bande

Le filtre biquad est un filtre numérique récursif de type IIR (Infinite Impulse Response) d'ordre deux, largement utilisé en traitement du signal pour son efficacité fréquentielle et son faible coût computationnel. Il repose sur une équation aux différences combinant les échantillons d'entrée actuels et passés ainsi que les sorties précédentes afin de produire un signal filtré.

Sa forme générale s'écrit :

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] - a_1y[n-1] - a_2y[n-2] \quad (2.2)$$

où $x[n]$ représente le signal d'entrée et $y[n]$ le signal filtré. Les coefficients b_i et a_i définissent la réponse fréquentielle du filtre.

Application dans le système de détection de pas

Dans notre application, le filtre biquad est utilisé en configuration passe-bande afin d'isoler la bande fréquentielle caractéristique de la marche humaine, définie entre :

$$f_1 = 0.6 \text{ Hz} \quad \text{et} \quad f_2 = 3.0 \text{ Hz} \quad (2.3)$$

Le filtrage est appliqué à la norme de l'accélération :

$$\text{mag}(t) = \sqrt{a_x^2 + a_y^2 + a_z^2} \quad (2.4)$$

La fréquence d'échantillonnage utilisée est fixée à :

$$f_s = 50 \text{ Hz} \quad (2.5)$$

Cette valeur a été choisie expérimentalement. Des tests réalisés avec cette fréquence ont montré un bon compromis entre précision fréquentielle, stabilité du signal filtré et performance temps réel, ce qui a conduit à conserver ce paramètre dans l'implémentation finale.

La fréquence centrale du filtre est définie par la moyenne géométrique des fréquences de coupure :

$$f_0 = \sqrt{f_1 * f_2} = \sqrt{0.6 * 3.0} = 1.3416 \text{ Hz} \quad (2.6)$$

Implémentation récursive

Le filtrage est effectué échantillon par échantillon selon la structure interne suivante :

$$y[n] = b_0 x[n] + z_1 \quad (2.7)$$

$$z_1 = b_1 x[n] - a_1 y[n] + z_2 \quad (2.8)$$

$$z_2 = b_2 x[n] - a_2 y[n] \quad (2.9)$$

où z_1 et z_2 représentent les états internes mémorisant l'historique du système.

Interprétation physique

Ce filtrage permet de supprimer les très basses fréquences, principalement associées à la gravité et aux variations lentes de posture, ainsi que les hautes fréquences dues aux chocs et au bruit du capteur. La bande dynamique propre aux oscillations de marche est ainsi conservée, ce qui rend les pics correspondant aux pas plus nets et plus facilement détectables.

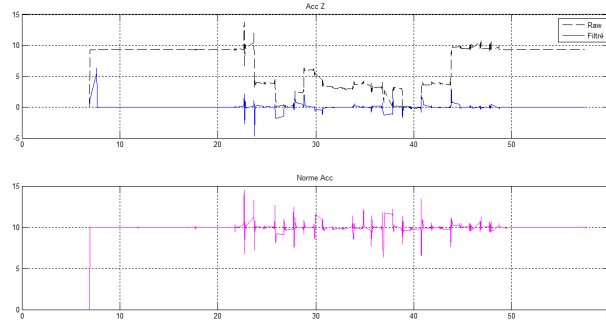


FIGURE 2.1 : en haut, visualisation du signal brut de l'accélération et du signal filtré, ainsi que de la norme de l'accélération en bas.

Le choix du filtre biquad se justifie par sa simplicité d'implémentation, son faible coût de calcul et sa stabilité numérique, le rendant particulièrement adapté à une application mobile de navigation fonctionnant en temps réel.

2.1.3 Filtrage du gyroscope

Le gyroscope mesure la vitesse angulaire du téléphone autour des trois axes. Dans notre application, seule la composante autour de l'axe vertical Z est utilisée pour la détection et la validation des virages, car elle correspond à la rotation horizontale du dispositif.

Le signal brut du gyroscope peut contenir du bruit haute fréquence ainsi que des micro-variations non représentatives d'un mouvement réel. Afin d'améliorer la stabilité de la mesure, un filtrage passe-bas de premier ordre est appliqué.

Filtrage exponentiel (IIR 1^{er} ordre)

Le filtrage est réalisé à l'aide d'un lissage exponentiel défini par :

$$\Omega_f[n] = (1 - \alpha) \Omega_f[n - 1] + \alpha \Omega_{\text{raw}}[n] \quad (2.10)$$

où :

- $\Omega_{\text{raw}}[n]$ est la vitesse angulaire brute mesurée par le capteur,
- $\Omega_f[n]$ est la valeur filtrée,
- α est le coefficient de lissage.

Ce filtre correspond à un système IIR causal de premier ordre, simple et peu coûteux en calcul.

Valeurs utilisées dans l'application

Deux configurations sont utilisées :

- En fonctionnement normal :

$$\alpha = 0.25$$

- Pendant le verrouillage de virage (Turn Lock) :

$$\alpha = 0.12$$

Une valeur plus faible de α pendant le verrouillage permet d'augmenter la stabilité du signal et de réduire l'influence des micro-oscillations lors de la validation d'un virage.

Interprétation physique

Le filtrage passe-bas permet d'atténuer les fluctuations rapides non significatives tout en conservant les variations lentes correspondant à une rotation réelle du corps.

Le signal filtré est ensuite utilisé pour :

- déterminer le sens de rotation (gauche ou droite),
- intégrer la vitesse angulaire afin d'estimer l'angle réellement tourné :

$$\theta = \int \Omega_f(t) dt$$

- valider physiquement qu'un virage a effectivement eu lieu.

Ce choix de filtrage constitue un compromis entre réactivité et stabilité, garantissant une détection fiable des rotations tout en maintenant une implémentation légère adaptée au fonctionnement temps réel sur smartphone.

2.1.4 Étalonnage et calibration utilisateur

Bien que la dynamique générale de la marche humaine présente une signature fréquentielle identifiable, l'allure spécifique des pas varie significativement d'un individu à un autre. La longueur de pas, la cadence, ainsi que l'amplitude du signal inertiel dépendent de plusieurs facteurs biomécaniques tels que la taille, la morphologie, la vitesse de marche ou encore le style locomoteur.

Afin de réduire l'erreur systématique accumulée en navigation inertielle, une phase d'étalonnage personnalisé est mise en place. Cette calibration permet d'estimer trois paramètres propres à l'utilisateur :

- la longueur moyenne du pas L_{pas} ,
- la période de marche de référence T_{ref} ,
- l'amplitude de référence du signal filtré A_{ref} .

Ces paramètres sont ensuite utilisés pour adapter dynamiquement la détection de pas et améliorer la précision de la navigation.

Principe général

L'utilisateur est invité à parcourir une distance connue fixée à :

$$D = 5 \text{ m}$$

Pendant ce déplacement, le système détecte les pas en temps réel à partir du signal d'accélération filtré (passe-bande 0.6–3 Hz).

Chaque candidat pas est validé uniquement s'il respecte des contraintes morphologiques et temporelles strictes :

- largeur du lobe positif comprise entre 120 ms et 400 ms,
- amplitude comprise entre 1.2 et 5.0 (unité normalisée),
- intervalle entre deux pas compris entre 450 ms et 3200 ms.

Ces contraintes permettent d'éliminer les perturbations et mouvements non liés à une marche physiologique normale.

Estimation robuste de la cadence

Les instants des pas validés sont stockés afin de calculer les intervalles :

$$\Delta t_i = t_i - t_{i-1}$$

La période de référence est estimée par la médiane des intervalles, ce qui garantit une robustesse face aux valeurs aberrantes :

$$T_{ref} = \text{median}(\Delta t_i)$$

Une fenêtre de validité autour de cette valeur est ensuite appliquée pour éliminer d'éventuels écarts excessifs.

Estimation de l'amplitude de référence

L'amplitude de chaque pas validé est enregistrée. L'amplitude de référence est définie comme :

$$A_{ref} = \text{median}(A_i)$$

Cette valeur sert ensuite à adapter dynamiquement les seuils de détection pendant la navigation.

Calcul de la longueur de pas

La distance connue $D = 5$ m est convertie en longueur de pas personnalisée en utilisant une pondération tenant compte de la cadence et de l'amplitude de chaque pas.

Chaque pas contribue avec un poids :

$$w_i = \left(\frac{T_{ref}}{\Delta t_i} \right)^{0.20} \left(\frac{A_i}{A_{ref}} \right)^{0.25}$$

La somme pondérée est :

$$W = \sum_i w_i$$

La longueur de pas est estimée à partir de la somme pondérée :

$$W = \sum_i w_i$$

puis :

$$L_{pas} = \frac{D}{W}$$

Cependant, dans certains cas exceptionnels, la somme pondérée W peut être nulle. Cela peut se produire si aucun pas valide n'a été retenu pendant la phase d'étalonnage, si les poids calculés ne sont pas exploitables (cas extrême ou anomalie de détection), ou encore si la procédure de calibration a échoué mais que la fonction de calcul est tout de même exécutée.

Afin de garantir la robustesse du système, une solution de repli est prévue. Si $W = 0$, la longueur de pas est alors estimée de manière simplifiée par :

$$L_{pas} = \frac{D}{N}$$

où N représente le nombre total de pas détectés.

Cette sécurité assure la continuité du fonctionnement du système, même dans des conditions non idéales.

Enfin, la valeur est contrainte dans un intervalle physiologique réaliste :

$$0.35 \text{ m} \leq L_{pas} \leq 0.95 \text{ m}$$

Interprétation physique

Cette procédure garantit :

- un ancrage métrique réel grâce à la distance connue
- une robustesse aux valeurs aberrantes via l’usage de médianes
- une adaptation biomécanique au style de marche individuel
- une réduction significative de la dérive en navigation

2.2 Calcul de l’orientation

2.2.1 Principe physique

Pour déterminer l’orientation de l’utilisateur dans le plan horizontal, on utilise les angles d’Euler, classiquement employés en navigation inertielle.

Un système d’orientation 3D peut être décrit par trois angles :

- le **roll** : rotation autour de l’axe longitudinal
- le **pitch** : inclinaison avant/arrière
- le **yaw** : rotation autour de l’axe vertical

Dans notre application, seul le yaw est utilisé, car il représente la direction horizontale vers laquelle l’utilisateur se dirige.

Physiquement, le yaw correspond à l’angle de rotation du téléphone autour de son axe vertical. Il représente l’orientation de l’axe avant du téléphone par rapport à une direction de référence définie par le système d’estimation d’attitude.

Dans notre cas, le yaw exprime la direction horizontale vers laquelle pointe le téléphone à un instant donné.

2.2.2 Capteurs utilisés

L’estimation de l’orientation repose principalement sur :

- le **Rotation Vector**, qui fournit directement une estimation fusionnée de l’orientation
- le **gyroscope**, utilisé principalement pour la validation dynamique des virages

2.2.3 Conversion en angles d’Euler

Le vecteur de rotation fourni par Android est converti en matrice de rotation, puis en angles d’Euler :

La matrice de rotation obtenue à partir du vecteur de rotation s'écrit :

$$\mathbf{R} = \begin{bmatrix} R_{11} & R_{12} & R_{13} \\ R_{21} & R_{22} & R_{23} \\ R_{31} & R_{32} & R_{33} \end{bmatrix} \quad (2.11)$$

Les angles d'Euler (yaw, pitch, roll) sont ensuite extraits de cette matrice selon :

$$\text{yaw} = \text{atan2}(R_{21}, R_{11}) \quad (2.12)$$

$$\text{pitch} = \arcsin(-R_{31}) \quad (2.13)$$

$$\text{roll} = \text{atan2}(R_{32}, R_{33}) \quad (2.14)$$

Le yaw brut est obtenu à partir de :

$$\theta_{raw} = \text{atan2}(R_{21}, R_{11}) \quad (2.15)$$

et ensuite converti en degrés

2.2.4 Normalisation angulaire

Les angles sont ramenés dans l'intervalle :

$$(-180^\circ, 180^\circ)$$

afin d'éviter les discontinuités au passage $359^\circ - 0^\circ$.

2.2.5 Filtrage circulaire

Un filtrage linéaire direct appliqué aux angles exprimés en degrés peut engendrer des erreurs artificielles lorsque l'angle traverse la discontinuité $\pm 180^\circ$. En effet, un passage naturel de 179° à -179° correspond à une faible rotation réelle, mais serait interprété numériquement comme une variation brutale de près de 358° , ce qui fausse le lissage.

Le système applique donc un filtrage circulaire :

$$s_n = (1 - \alpha)s_{n-1} + \alpha \sin(\theta_{raw}) \quad (2.16)$$

$$c_n = (1 - \alpha)c_{n-1} + \alpha \cos(\theta_{raw}) \quad (2.17)$$

$$\theta_{smooth} = \text{atan2}(s_n, c_n) \quad (2.18)$$

avec $\alpha = 0.08$.

Cette méthode permet un lissage stable sans rupture artificielle autour de $\pm 180^\circ$.

2.2.6 Déroulement continu (Unwrap)

Déroulement continu (Unwrap)

Le yaw fourni par le système est toujours normalisé dans l'intervalle $(-180^\circ, 180^\circ)$.

Cela signifie que lorsqu'on dépasse 180° , l'angle revient brutalement à -180° . Par exemple :

$$179^\circ \rightarrow -179^\circ$$

Numériquement, la différence directe serait :

$$-179^\circ - 179^\circ = -358^\circ$$

alors qu'en réalité le téléphone n'a tourné que de $+2^\circ$.

Pour éviter cette erreur artificielle, on ne travaille pas avec la différence brute. On calcule d'abord la plus petite variation angulaire entre deux instants :

$$\Delta\theta = \text{wrap}_{(-180^\circ, 180^\circ]}(\theta_k - \theta_{k-1})$$

Cette opération garantit que $\Delta\theta$ est toujours compris entre -180° et $+180^\circ$, et correspond à la rotation réelle minimale.

Ensuite, on reconstruit un angle cumulatif continu :

$$\theta_{\text{abs}}(k) = \theta_{\text{abs}}(k-1) + \Delta\theta$$

Ainsi, si l'utilisateur tourne progressivement, l'angle peut évoluer comme :

$$170^\circ \rightarrow 179^\circ \rightarrow 181^\circ \rightarrow 190^\circ$$

même si l'angle normalisé passe temporairement par -179° .

Limitation physique

Dans l'implémentation, la variation angulaire est également limitée par une vitesse maximale :

$$|\Delta\theta| \leq 140^\circ/s \cdot \Delta t$$

La valeur de la vitesse angulaire maximale a été déterminée expérimentalement à l'issue de plusieurs phases de test. Elle dépend directement de la fréquence d'échantillonnage du capteur utilisé (50Hz dans notre cas)

Cette contrainte garantit que les variations instantanées restent physiquement plausibles et évite les sauts dus au bruit capteur.

Ainsi, l'algorithme suit fidèlement la rotation réelle du téléphone dans le temps, sans discontinuité artificielle liée aux bornes angulaires.

2.2.7 Passage au repère carte

L'orientation du téléphone doit être alignée avec le repère de la carte. Un décalage est donc introduit :

$$\theta_{rel} = \theta_{abs} - \theta_{offset}$$

Ce décalage est initialisé lors du démarrage de la navigation afin d'aligner l'utilisateur avec la direction du premier segment du chemin.

2.2.8 Rôle dans la navigation

Le yaw final filtré est utilisé pour :

- comparer l'orientation actuelle à celle du segment courant
- déterminer si l'utilisateur est aligné ou non
- valider les virages en combinaison avec le gyroscope

Ainsi, un algorithme de planification de trajectoire détermine la direction théorique à suivre, tandis que le yaw permet de vérifier en temps réel que l'utilisateur est orienté correctement.

2.3 Planification de trajectoire par algorithme A*

2.3.1 Contexte et motivation

Une fois la détection des pas rendue robuste et la longueur de pas personnalisée estimée, il devient possible de transformer les déplacements détectés en progression métrique fiable.

La prochaine étape consiste à guider l'utilisateur dans le bâtiment à partir d'un plan réel fourni (plan de l'institut), représentant les murs, les portes, les escaliers et les zones accessibles.

L'objectif est de permettre à l'utilisateur de choisir librement un point de départ et un point d'arrivée, puis de calculer automatiquement le chemin praticable reliant ces deux points.

Plusieurs méthodes de planification existent en robotique mobile. Après analyse, l'algorithme A* a été retenu en raison de sa rapidité, de sa robustesse et de son efficacité dans les environnements discrets avec obstacles.

2.3.2 Représentation de la carte

Le plan du bâtiment est transformé en grille discrète.

Chaque cellule peut être :

- marchable (sol, portes)
- non marchable (murs, obstacles)

La discrétisation est effectuée avec un pas spatial de 6 pixels. Cette représentation simplifie les calculs et permet l'utilisation d'un algorithme de recherche sur graphe.

2.3.3 Principe de l'algorithme A*

L'algorithme A* est une méthode de recherche de chemin optimal sur une grille.

Chaque position explorée est appelée nœud et possède :

- un coût réel déjà parcouru : g
- une estimation de la distance restante au but : h
- un score total :

$$f = g + 1.2 h$$

Le terme g représente le coût cumulé du chemin depuis le départ. Le terme h correspond à la distance euclidienne jusqu'à la destination.

Le facteur 1.2 introduit une pondération (Weighted A*) qui favorise les chemins se dirigeant plus directement vers le but, accélérant la convergence au prix d'une très légère perte d'optimalité théorique.

2.3.4 Voisinage et coûts

Chaque cellule possède 8 voisins :

- 4 directions principales (haut, bas, gauche, droite), coût = 1,
- 4 diagonales, coût = 1.414.

Ainsi, le coût cumulé g reflète une distance géométrique cohérente sur la grille.

2.3.5 Recherche et arrêt

L'algorithme maintient deux ensembles :

- une liste ouverte (nœuds à explorer, triés selon f),
- une liste fermée (nœuds déjà évalués).

À chaque étape, le nœud le plus prometteur (plus petit score f) est exploré. La recherche s'arrête lorsque la distance au but devient inférieure ou égale au pas spatial de la grille.

2.3.6 Reconstruction et lissage du chemin

Une fois la destination atteinte, le chemin est reconstruit en remontant les pointeurs parents.

Un post-traitement est ensuite appliqué :

- suppression des points intermédiaires inutiles si une ligne droite est libre
- calcul de la distance cumulée le long de la polyligne

Ce lissage supprime les effets en "escaliers" dus à la grille et produit un trajet plus naturel.

2.3.7 Intégration avec la navigation inertielle

Le chemin obtenu constitue une contrainte géométrique fixe. La position de l'utilisateur n'est pas estimée librement dans le plan (dead-reckoning pur). À chaque pas détecté, la progression est projetée le long de la polyligne A^* .

Ainsi :

- la dérive latérale est fortement réduite
- le déplacement reste physiquement cohérent avec la structure du bâtiment
- les virages sont détectés par comparaison des orientations successives des segments

2.3.8 Gestion et validation des virages

Lorsque l'algorithme A^* génère un chemin présentant un changement d'orientation significatif entre deux segments successifs, un événement de virage est identifié.

Détection géométrique du virage

Pour chaque paire de segments consécutifs du chemin lissé, on calcule :

$$\Delta\theta = \text{wrap}_{(-\pi, \pi]}(\theta_2 - \theta_1) \quad (2.19)$$

où $\text{wrap}_{(-\pi, \pi]}(\cdot)$ désigne l'opérateur ramenant un angle dans l'intervalle $(-\pi, \pi)$ afin d'éviter les discontinuités angulaires.

Si :

$$|\Delta\theta| \geq 60^\circ$$

alors un virage est déclaré. Le signe de $\Delta\theta$ permet de déterminer la direction :

- $\Delta\theta > 0$: virage à droite
- $\Delta\theta < 0$: virage à gauche

La position du virage est définie à partir de la distance cumulée le long du chemin.

Activation du verrouillage (Turn Lock)

Lorsque l'utilisateur approche du point de virage, un mécanisme de verrouillage est activé.

Ce verrouillage a pour objectif :

- de suspendre temporairement l'avancement normal basé uniquement sur les pas,
- de surveiller la rotation réelle du téléphone,
- de vérifier que le virage prévu par la carte est effectivement exécuté.

Pendant cette phase :

- les nouveaux pas détectés sont temporairement mis en attente,
- une direction de rotation attendue (gauche ou droite) est fixée,
- une orientation cible (yaw du nouveau segment) est définie.

Validation physique du virage

La validation repose sur une double vérification inertielle.

1. Validation gyroscopique La vitesse angulaire autour de l'axe vertical est mesurée Ω_z

Après filtrage, le signe est ajusté en fonction de la direction attendue :

$$\Omega_{\text{valid}} = \Omega_z \cdot s_{\text{attendu}}$$

où $s_{\text{attendu}} \in \{+1, -1\}$ dépend du sens de rotation attendu (gauche/droite).

Ainsi :

- rotation dans le bon sens donne une valeur positive
- rotation dans le mauvais sens donne une valeur négative

L'angle réellement tourné est obtenu par intégration :

$$\theta = \int \Omega_{valid}(t) dt$$

Un seuil minimal d'angle tourné est exigé afin d'éviter les faux virages dus au bruit.

2. Validation directionnelle (yaw) En parallèle, l'orientation finale mesurée (yaw) doit s'aligner avec la direction du nouveau segment :

$$|\theta_{yaw} - \theta_{cible}| \leq \text{tolérance}$$

Cette condition doit être vérifiée pendant plusieurs cycles successifs afin d'assurer la stabilité.

Cas possibles

- Cas normal : rotation détectée, angle suffisant, alignement correct alors on valide le virage et on reprend la navigation.
- Mouvement insuffisant ou bruit : pas de validation, maintien du verrouillage.
- Timeout : réinitialisation partielle afin d'éviter un blocage permanent.

Interprétation globale

L'algorithme A* détermine géométriquement où un virage doit être effectué. Le gyroscope et le yaw vérifient physiquement si le virage a réellement été exécuté.

Cette double validation, géométrique et inertielle, renforce considérablement la robustesse du système en réduisant les faux positifs et les erreurs d'orientation.

Fonctionnement global du système

Le système de navigation développé repose sur une architecture hybride combinant navigation inertielle piétonne (PDR), traitement du signal embarqué et planification géométrique de trajectoire.

Son objectif est de transformer les mesures physiques issues des capteurs d'un smartphone en une estimation cohérente, stable et temps réel de la position et de l'orientation de l'utilisateur à l'intérieur d'un bâtiment.

1. Principe physique général

Le système exploite l'unité de mesure inertielle (IMU) intégrée au smartphone :

- Accéléromètre : mesure des accélérations linéaires

- Gyroscope : mesure des vitesses angulaires
- Rotation Vector : estimation fusionnée de l'orientation 3D

Deux grandeurs principales sont estimées :

- la translation, obtenue par détection des pas et estimation de la longueur de pas
- la rotation, obtenue par estimation du yaw (orientation horizontale)

La mise à jour de la position s'effectue de manière discrète à chaque pas détecté selon :

$$\mathbf{p}_{k+1} = \mathbf{p}_k + L_{pas} \begin{bmatrix} \cos(\theta_k) \\ \sin(\theta_k) \end{bmatrix}$$

où :

- $\mathbf{p}_k = \begin{bmatrix} x_k \\ y_k \end{bmatrix}$ représente la position estimée de l'utilisateur au pas numéro k
- $\mathbf{p}_{k+1}$ représente la position après le pas suivant
- L_{pas} est la longueur de pas calibrée de l'utilisateur
- θ_k est le yaw filtré à l'instant k , c'est-à-dire l'orientation horizontale du téléphone
- k désigne l'indice discret correspondant au nombre de pas détectés

Physiquement, cette équation signifie qu'à chaque pas, la position précédente est incrémentée d'un vecteur de norme L_{pas} orienté selon la direction donnée par le yaw.

Ainsi, le déplacement est modélisé comme une succession de translations élémentaires alignées avec l'orientation instantanée de l'utilisateur.

2. Chaîne de traitement globale

Acquisition capteurs

Les signaux inertiels sont acquis à une fréquence de :

$$f_s = 50 \text{ Hz}$$

Les grandeurs mesurées sont :

- a_x, a_y, a_z (accélérations)
- Ω_z (vitesse angulaire verticale)

Traitement du signal

Plusieurs filtrages sont appliqués :

- Filtre biquad passe-bande (0.6–3 Hz) pour isoler la dynamique de marche
- Lissage exponentiel du gyroscope pour stabiliser la rotation mesurée
- Filtrage circulaire du yaw pour éviter les discontinuités $\pm 180^\circ$
- Déroulement angulaire (unwrap) pour obtenir un angle continu

Ces traitements garantissent une cohérence physique du mouvement estimé

3. Calibration personnalisée

Une phase d'étalonnage sur une distance connue :

$$D = 5 \text{ m}$$

permet d'estimer :

- la longueur moyenne du pas L_{pas}
- la période de marche T_{ref}
- l'amplitude de référence A_{ref}

La longueur de pas est calculée par :

$$L_{pas} = \frac{D}{\sum_i w_i}$$

avec :

$$w_i = \left(\frac{T_{ref}}{\Delta t_i} \right)^{0.20} \left(\frac{A_i}{A_{ref}} \right)^{0.25}$$

Une contrainte physiologique est ensuite appliquée :

$$0.35 \text{ m} \leq L_{pas} \leq 0.95 \text{ m}$$

4. Gestion des unités : pixels et mètres

Le plan du bâtiment est représenté en pixels, tandis que la navigation inertielle fonctionne en mètres.

Une constante d'échelle est utilisée :

$$PX_PER_M = 30$$

Elle définit la correspondance :

$$px = m * PX_PER_M$$

$$m = \frac{px}{PX_PER_M}$$

Ainsi :

- La longueur de pas en pixels est donnée par :

$$L_{pas}^{px} = L_{pas} * PX_PER_M$$

- La distance totale du chemin est convertie par :

$$d_m = \frac{d_{px}}{PX_PER_M}$$

Tout le calcul géométrique du chemin est effectué en pixels, puis converti en mètres pour la cohérence avec la PDR.

5. Planification par A*

Le plan est discrétisé en grille (pas spatial 6 px).

Chaque cellule peut être :

- marchable
- non marchable (mur)

L'algorithme A* pondéré utilise :

$$f = g + 1.2h$$

où :

- g est le coût réel parcouru
- h la distance euclidienne au but

Le chemin obtenu est ensuite lissé pour supprimer les effets de grille.

6. Navigation temps réel

À chaque pas validé, le système applique une translation élémentaire :

$$\Delta s = L_{pas}$$

où :

- Δs représente le déplacement élémentaire effectué lors d'un pas,
- L_{pas} est la longueur de pas calibrée et personnalisée de l'utilisateur.

Autrement dit, chaque pas détecté correspond à un déplacement d'une distance fixe égale à la longueur moyenne estimée lors de l'étalonnage.

Cependant, la position n'est pas calculée par une intégration libre dans le plan (dead-reckoning pur), ce qui entraînerait une accumulation progressive d'erreurs.

Le déplacement estimé est au contraire contraint à suivre la polygline obtenue par l'algorithme A*. Concrètement, la progression est projetée le long du chemin calculé sur la carte.

Cette contrainte géométrique permet de limiter fortement la dérive latérale et d'assurer que la trajectoire reste cohérente avec la structure réelle du bâtiment.

7. Gestion des virages

Un virage est détecté géométriquement si :

$$|\Delta\theta| \geq 60^\circ$$

Lorsqu'on approche d'un virage :

- activation d'un verrouillage
- intégration de la rotation gyroscopique :

$$\theta = \int \Omega(t) dt$$

- vérification d'alignement du yaw avec la nouvelle direction.

La validation est double :

- preuve inertielle (rotation réelle)
- preuve directionnelle (orientation correcte)

8. Architecture globale

Le système combine :

- estimation inertielle locale (pas + yaw)
- calibration biomécanique personnalisée
- planification géométrique A*
- contrainte spatiale sur la carte

Il en résulte un système hybride robuste, adapté à la navigation intérieure temps réel, fonctionnant uniquement avec les capteurs embarqués du smartphone, sans infrastructure externe.

Conclusion

Ce chapitre représente l'architecture complète du système de navigation intérieure, depuis l'acquisition des mesures IMU jusqu'à la prise de décision en temps réel. La détection de pas filtrée et calibrée, associée à l'estimation robuste du yaw, permet une reconstruction cohérente du mouvement de l'utilisateur. Enfin, la planification A* et la contrainte de trajectoire, renforcées par la validation inertielle des virages, assurent une navigation stable et fiable dans le bâtiment.

CONCEPTION ET ARCHITECTURE DE LA SOLUTION PROPOSÉE

Plan

1	Vue d'ensemble de la solution de navigation indoor	31
2	Architecture globale du système	32
3	Logique de navigation et enchaînement des traitements	34
4	Gestion intelligente des virages	40
5	Interaction utilisateur et retour visuel	44
6	Communication avec MATLAB et outils d'analyse	44

Introduction

Ce chapitre est consacré à la présentation de l'architecture de la solution de navigation indoor développée. Il décrit d'abord l'architecture globale du projet afin de donner une vision générale du système. Ensuite, l'architecture interne de l'application est présentée pour expliquer son organisation. Enfin, les différents modules mis en œuvre sont expliqués afin de montrer comment ils coopèrent pour réaliser la solution proposée.

3.1 Vue d'ensemble de la solution de navigation indoor

La solution proposée vise à assurer une navigation en environnement intérieur sans recourir au système GPS, dont l'utilisation devient inefficace à l'intérieur des bâtiments. Elle repose sur l'exploitation des capteurs intégrés au smartphone et sur l'utilisation d'un plan intérieur afin de guider l'utilisateur de manière progressive entre un point de départ et un point d'arrivée.

Le principe général de fonctionnement de la solution est basé sur la combinaison de plusieurs éléments. Tout d'abord, un plan intérieur du bâtiment est utilisé comme support de navigation. Ensuite, les déplacements de l'utilisateur sont estimés à partir de la détection des pas, tandis que l'orientation du téléphone permet de déterminer la direction du mouvement. En combinant ces informations, la position de l'utilisateur est mise à jour progressivement le long d'un trajet préalablement calculé, ce qui permet d'assurer un suivi continu du déplacement en environnement intérieur.

Le smartphone joue un rôle central dans cette solution. Il assure l'acquisition des données issues des capteurs inertiels, notamment pour la détection des mouvements et de l'orientation. Il réalise également le traitement de ces données afin d'estimer le déplacement de l'utilisateur et de gérer la logique de navigation. En parallèle, le smartphone est responsable de l'affichage de la carte, de l'itinéraire et des indications de navigation à l'utilisateur. Enfin, il permet la communication avec l'environnement MATLAB, utilisé pour l'analyse et la visualisation des données en temps réel.

Le fonctionnement global de la solution s'articule autour de plusieurs étapes successives. Une première étape d'étalonnage est réalisée afin d'adapter le système aux caractéristiques de marche de l'utilisateur. L'utilisateur sélectionne ensuite un point de départ et une destination sur le plan intérieur. À partir de ces informations, un chemin optimal est calculé. La navigation débute alors et s'effectue pas à pas, avec une mise à jour continue de la position et une gestion spécifique des changements de direction et des virages afin de garantir une navigation fluide et cohérente.



FIGURE 3.1 : Vue générale du fonctionnement de la solution proposée

3.2 Architecture globale du système

Dans cette partie, nous présentons l'architecture globale du système proposé, en commençant par l'architecture logicielle générale, puis en détaillant le découpage fonctionnel de l'application Android.

3.2.1 Architecture logicielle générale

L'architecture globale du système a été conçue de manière simple et modulaire afin d'assurer un fonctionnement fluide et une bonne lisibilité de la solution proposée. Elle repose principalement sur une application mobile Android, qui constitue le cœur du système de navigation indoor développé dans le cadre de ce projet.

L'application mobile prend en charge l'ensemble des traitements nécessaires à la navigation. Elle exploite les données issues des capteurs inertiels du smartphone (IMU) afin d'estimer les déplacements et l'orientation de l'utilisateur, puis affiche en temps réel les informations de guidage nécessaires à la navigation. Afin de faciliter l'analyse du comportement du système et la validation des résultats obtenus, une interaction avec l'environnement MATLAB a également été mise en place. Cette interaction permet d'observer et de visualiser certaines données de navigation en temps réel, notamment à des fins de débogage.

La communication entre l'application Android et l'environnement MATLAB est assurée à l'aide du protocole UDP. Ce choix permet un échange rapide des données, adapté aux contraintes du temps réel, tout en conservant une architecture légère. Il est important de souligner que l'application reste pleinement fonctionnelle et autonome, même en l'absence de cette communication externe.

Cette organisation permet ainsi de distinguer clairement les fonctions de navigation embarquées

sur le smartphone des outils d'analyse et de visualisation utilisés pour l'évaluation et le débogage du système, contribuant ainsi à une architecture claire et bien structurée.

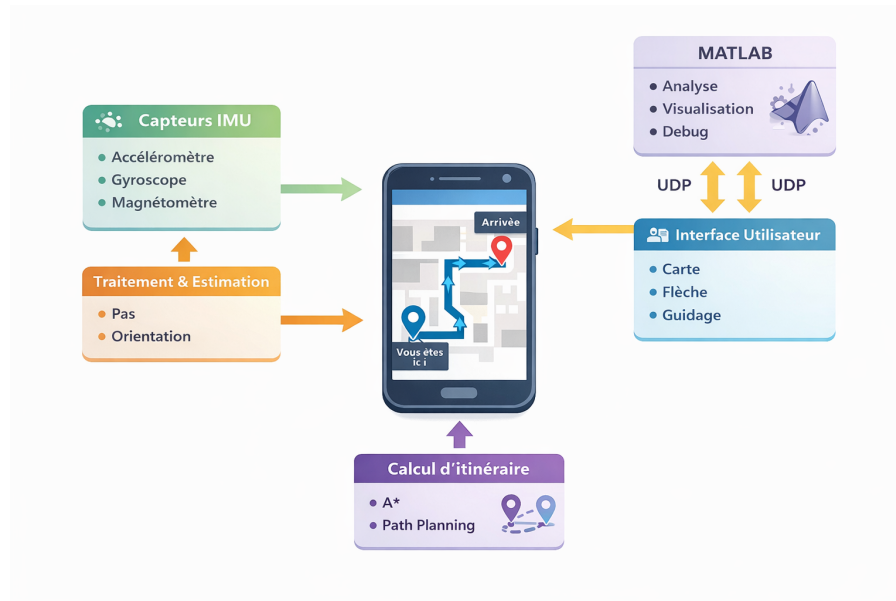


FIGURE 3.2 : Architecture logicielle de l'application de navigation indoor

3.2.2 Découpage fonctionnel de l'application Android

Afin de garantir une bonne organisation et une meilleure compréhension du fonctionnement global, l'application Android a été conçue selon un découpage fonctionnel en plusieurs sous-parties, chacune ayant un rôle bien défini. Cette approche permet de structurer l'application de manière claire tout en assurant une coopération efficace entre les différents éléments de la solution.

La première partie fonctionnelle est dédiée à l'acquisition des données. Elle permet de récupérer les informations fournies par les capteurs inertiels du smartphone, nécessaires à l'estimation des mouvements et de l'orientation de l'utilisateur. Ces données constituent la base du fonctionnement de la navigation indoor.

Une autre partie fonctionnelle est consacrée au traitement des déplacements et de l'orientation. Elle exploite les données acquises afin d'estimer la progression de l'utilisateur, de suivre sa direction de déplacement et de détecter les changements de direction, notamment lors des virages. Cette étape permet d'assurer une mise à jour cohérente de la position estimée sur le plan intérieur.

L'application intègre également une partie dédiée à la gestion de la navigation. Celle-ci prend en charge le calcul du chemin entre le point de départ et la destination, ainsi que le suivi progressif de l'utilisateur le long de cet itinéraire. Elle permet d'assurer un guidage pas à pas tout au long du déplacement.

Une autre sous-partie est responsable de l'interface utilisateur. Elle permet l'affichage du plan intérieur, du trajet calculé et des indications de navigation, tout en assurant l'interaction avec l'utilisateur

pour la sélection des différents paramètres de navigation. Cette partie vise à offrir une interface simple, claire et intuitive.

Enfin, une partie fonctionnelle est dédiée à la communication avec l'environnement MATLAB. Elle permet l'envoi de certaines données de navigation afin de faciliter l'analyse, la visualisation et le débogage du système, sans perturber le fonctionnement principal de l'application.

Ce découpage fonctionnel permet ainsi de structurer l'application Android de manière cohérente, en séparant clairement les responsabilités de chaque sous-partie, tout en assurant une collaboration efficace pour la réalisation de la solution de navigation indoor proposée.



FIGURE 3.3 : Architecture fonctionnelle de l'application de navigation indoor

3.3 Logique de navigation et enchaînement des traitements

Cette partie a pour objectif de décrire la logique générale de navigation mise en œuvre dans l'application ainsi que l'enchaînement des différents traitements réalisés pendant l'exécution. Elle présente, de manière progressive, les principales étapes permettant de préparer la navigation, de suivre la progression de l'utilisateur et de gérer les situations spécifiques rencontrées lors du déplacement. Cette approche permet de mieux comprendre le fonctionnement interne de la solution et sert de base à l'explication détaillée des différentes parties du code.

3.3.1 Initialisation et prérequis de navigation

Avant de démarrer la navigation, l'application passe par une phase d'initialisation indispensable. Cette étape permet de préparer l'environnement de navigation et de s'assurer que toutes les données nécessaires sont disponibles et cohérentes avant le lancement du guidage.

```

// ===== RESET =====
1 Usage
private fun resetNavigationState(keepPoints: Boolean) {
    navigationActive = false
    navState = NavState.IDLE
    navPaused = false
    startConfirmed = false
    endConfirmed = false
    yawOffsetLockedToPath = false
    pendingStepsWhileLocked = 0
    yawSmoothPrevDeg = null
    yawAbsContDeg = 0f
    yawOffsetContDeg = 0f
    closeCsvLogger()
}

```

FIGURE 3.4 : Configuration initiale du module de navigation

```

// ===== LIFE CYCLE =====
@v override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    // ---- Charge l'étalonnage utilisateur (si existant) ----
    run {
        val prefs = getSharedPreferences(PREFS_NAME, MODE_PRIVATE)
        stepLenMUser = prefs.getFloat(key = PREF_STEP_LEN_M, defValue = STEP_LEN_M_DEFAULT)
        stepRefLenMUser = stepLenMUser
        stepRefPeriodMsUser = prefs.getFloat(key = PREF_STEP_REF_PERIOD_MS, defValue = stepRefPeriodMs)
        stepAmpRefUser = prefs.getFloat(key = PREF_STEP_AMP_REF, defValue = stepAmpRefUser)
        stepPeriodMs = stepRefPeriodMsUser
        Log.i(
            TAG,
            msg = "Loaded stepLenMUser=%.3f stepRefPeriodMsUser=%.0f"
                .format(Locale.US, ...args = stepLenMUser, stepRefPeriodMsUser)
        )
    }
}

```

FIGURE 3.5 : Chargement des paramètres utilisateur au démarrage

Lors de cette phase, l'application commence par charger les éléments liés à l'environnement intérieur, notamment le plan du bâtiment et les informations associées aux zones accessibles. Cette préparation permet de disposer d'un support cartographique fiable sur lequel le trajet sera calculé et suivi par la suite. En parallèle, les structures de données nécessaires à la navigation sont initialisées afin de garantir un fonctionnement fluide lors de l'exécution.


```

class MainActivity : AppCompatActivity(), SensorEventListener {
    override fun onCreate(savedInstanceState: Bundle?) {
        scaleDetector = ScaleGestureDetector(context = this, listener = object {
            // ...
        })
    }

    // Charge plan (modifiable pour doors)
    val mapOpts = BitmapFactory.Options().apply {
        inPreferredConfig = Bitmap.Config.ARGB_8888
        inMutable = true
    }
    bmpPlan = BitmapFactory.decodeResource(
        resources,
        id = com.example.sensortomatlab.R.drawable.rdc_galilee,
        mapOpts
    )
    Thread { detectAndDilateDoors() }
    // Base immuable pour affichage (après doors)

    // Overlay utilisable

```

FIGURE 3.6 : Chargement du plan du bâtiment et préparation des zones navigables

```

private fun precomputeWalkable() {
    val w = bmpPlan.width
    val h = bmpPlan.height
    for (y in 0 until h) for (x in 0 until w) {
        val k = y * w + x
        val c = bmpPlan.getPixel(x, y)
        walkable[k] = doorMask.contains(k) ||
            (Color.red(c) > 240 && Color.green(c) > 240 && Color.blue(c) > 240)
    }
}

private fun isWalkable(x: Int, y: Int): Boolean {
    if (!walkable.isInitialized) return false
    if (x !in 0 until bmpPlan.width || y !in 0 until bmpPlan.height) return false
    return walkable[y * bmpPlan.width + x]
}

```

FIGURE 3.7 : Implémentation des fonctions de détection et validation des zones accessibles

Un point essentiel de cette étape concerne la récupération des paramètres issus de l'étalonnage de l'utilisateur. Ces paramètres, obtenus lors d'une phase préalable, permettent d'adapter le système aux caractéristiques de marche propres à chaque utilisateur, telles que la longueur de pas et la cadence. Ils jouent un rôle central dans l'estimation du déplacement et sont utilisés tout au long de la navigation pour améliorer la précision du suivi.

```
// ---- Charge l'étalonnage utilisateur (si existant) ----
run {
    val prefs = getSharedPreferences(PREFS_NAME, MODE_PRIVATE)
    stepLenMUser = prefs.getFloat( key = PREF_STEP_LEN_M, defValue = STEP_LEN_M_DEFAULT)
    stepRefLenMUser = stepLenMUser
    stepRefPeriodMsUser = prefs.getFloat( key = PREF_STEP_REF_PERIOD_MS, defValue = stepRefPeriodMsDefault)
    stepAmpRefUser = prefs.getFloat( key = PREF_STEP_AMP_REF, defValue = stepAmpRefUser)
    stepPeriodMs = stepRefPeriodMsUser
    Log.i(
        TAG,
        msg = "Loaded stepLenMUser=%.3f stepRefPeriodMsUser=%.0f"
            .format(Locale.US, ...args = stepLenMUser, stepRefPeriodMsUser)
    )
}
```

FIGURE 3.8 : Chargement des paramètres utilisateur

L'application procède également à l'initialisation des différents modules nécessaires au fonctionnement de la navigation, notamment ceux liés aux capteurs, à la gestion de la navigation et à l'affichage. Cette étape garantit que les données pourront être traitées en temps réel dès le début du déplacement.

```
class MainActivity : AppCompatActivity(), SensorEventListener {
    override fun onCreate(savedInstanceState: Bundle?) {
        Thread { detectAndDilateDoors();

        // Buffer walkable

        sensorManager = getSystemService( name = SENSOR_SERVICE) as SensorManager
        accelerometer = sensorManager.getDefaultSensor( type = Sensor.TYPE_ACCELEROMETER)

        // IMPORTANT : on préfère ROTATION_VECTOR (avec mag) si dispo -> yaw moins précis à mesurer
        val rot = sensorManager.getDefaultSensor( type = Sensor.TYPE_ROTATION_VECTOR)
        val game = sensorManager.getDefaultSensor( type = Sensor.TYPE_GAME_ROTATION_VECTOR)
        rotationVector = rot
        gameRotationVector = game
        gyroscope = sensorManager.getDefaultSensor( type = Sensor.TYPE_GYROSCOPE)
        magnetometer = sensorManager.getDefaultSensor( type = Sensor.TYPE_MAGNETIC_FIELD)
        usingGameRV = false

        Log.i(
            TAG,
            msg = "RotationVector=${rotationVector?.name} GameRV=${gameRotationVector?.name} gyro=${gyro}"
        )
    }
}
```

FIGURE 3.9 : Initialisation des capteurs IMU

```
override fun onCreate(savedInstanceState: Bundle?) {
    Thread { detectAndDilateDoors();
        TAG,
        msg = "RotationVector=${rotationVector?.name} GameRV=${gameRotationVector?.name} gyro=${gyro}"
    )

    vibrator = getSystemService( name = VIBRATOR_SERVICE) as Vibrator

    try {
        socketSend = DatagramSocket().apply { soTimeout = 0 }
        socketRecv = DatagramSocket(portRecv).apply { soTimeout = 1000 } // timeout pour sortir
    } catch (e: Exception) {
        Log.e(TAG, msg = "UDP socket error", tr = e)
    }

    udpSendLine("CTRL,PARAM,PXPERM,%.2f" .format(Locale.US, ...args = PX_PER_M))
    udpSendLine("CTRL,PARAM,STEPLEN,%.3f" .format(Locale.US, ...args = stepLenMUser))
}
```

FIGURE 3.10 : Vérification des conditions préalables au lancement de la navigation

Enfin, avant de lancer la navigation, certaines conditions doivent être vérifiées. L'utilisateur doit avoir défini un point de départ et une destination sur le plan intérieur. Une fois ces éléments validés et les paramètres nécessaires chargés, l'application peut lancer le calcul du chemin optimal.

Cette transition marque le passage entre la phase de préparation et le début effectif de la navigation.

```

    }
    row1.setOnTouchListener(navChildTouchListener)
    row2.setOnTouchListener(navChildTouchListener)
    navHintText.setOnTouchListener(navChildTouchListener)
    pickStartButton.setOnTouchListener(navChildTouchListener)
    confirmButton.setOnTouchListener(navChildTouchListener)
    pickEndButton.setOnTouchListener(navChildTouchListener)
    restartButton.setOnTouchListener(navChildTouchListener)
    pauseResumeButton.setOnTouchListener(navChildTouchListener)
    row1.addView( child = pickStartButton, params = btnLp)
    row1.addView( child = pickEndButton, params = btnLp)

```

FIGURE 3.11 : Configuration des interactions utilisateur et des commandes de navigation

3.3.2 Définition du trajet et préparation de la navigation

Une fois la phase d'initialisation terminée et les prérequis validés, l'application passe à la définition du trajet à suivre. Cette étape a pour objectif de transformer les choix de l'utilisateur (point de départ et destination) en un chemin exploitable par le système de navigation.

La première action consiste à récupérer les positions sélectionnées par l'utilisateur sur le plan intérieur. Ces positions sont ensuite adaptées à l'environnement réel du bâtiment afin de garantir qu'elles correspondent à des zones accessibles. Cette étape permet d'éviter le calcul de trajets passant par des zones non marchables.

À partir de ces points, l'application lance le calcul du chemin optimal reliant le point de départ à la destination. Ce calcul s'appuie sur le plan intérieur et sur les zones accessibles préalablement définies. Le résultat obtenu est un ensemble ordonné de points représentant le trajet que l'utilisateur devra suivre.

```

private fun computePathAsync() {
    val s0 = startPoint ?: return
    val e0 = endPoint ?: return
    val s = snapToWalkable( p = s0)
    val e = snapToWalkable( p = e0)

    pathExec.execute {
        try {
            // 1) A* path on walkable grid
            val raw = runAStar(s, e)
            if (raw.size < 2) {
                Log.w(TAG, msg = "A* returned empty/too short path")
                showToast( message = "A* no path", length = Toast.LENGTH_LONG)
                return@execute
            }

            // 2) Optional smoothing
            val computed = smoothPath(raw)
            if (computed.size < 2) {

```

FIGURE 3.12 : Implémentation du calcul du chemin et vérification de validité

Une fois le chemin calculé, plusieurs informations nécessaires à la navigation sont préparées. La longueur totale du trajet est estimée et des structures de données sont mises en place afin de permettre

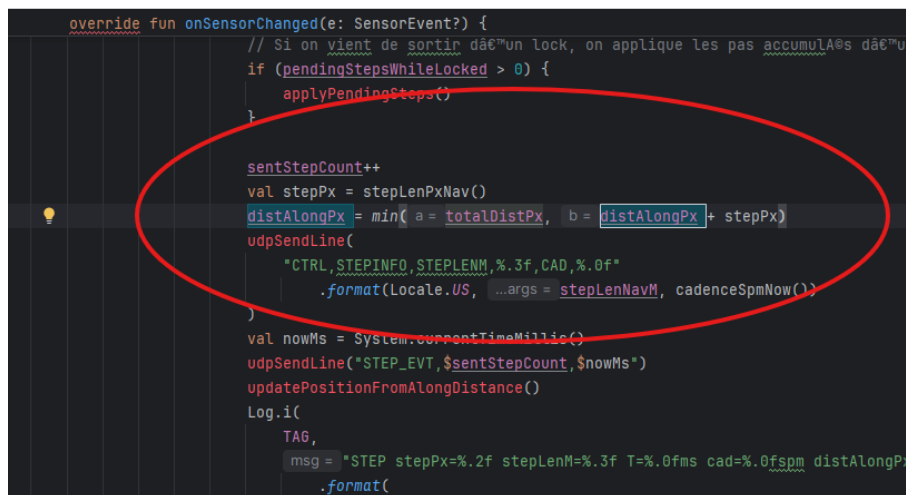
le suivi progressif de la position de l'utilisateur le long de ce chemin. Cette préparation est essentielle pour assurer une navigation continue et cohérente lors du déplacement.

Enfin, l'application termine cette étape en mettant à jour l'état de la navigation. Le système est alors prêt à démarrer le suivi en temps réel de l'utilisateur et à gérer dynamiquement sa progression sur le trajet calculé.

3.3.3 Suivi de la progression et mise à jour de la position de l'utilisateur sur le trajet

Une fois le trajet calculé et la navigation lancée, l'application assure le suivi continu de la progression de l'utilisateur le long du chemin défini. Cette étape constitue le cœur de la navigation indoor, car elle permet de mettre à jour en temps réel la position estimée de l'utilisateur en fonction de ses déplacements réels.

Le suivi de la progression repose principalement sur la détection des pas de l'utilisateur. À chaque pas détecté, une distance estimée est ajoutée à la progression globale, en tenant compte des paramètres d'étalonnage précédemment chargés. Cette approche permet d'estimer de manière progressive la distance parcourue depuis le point de départ.



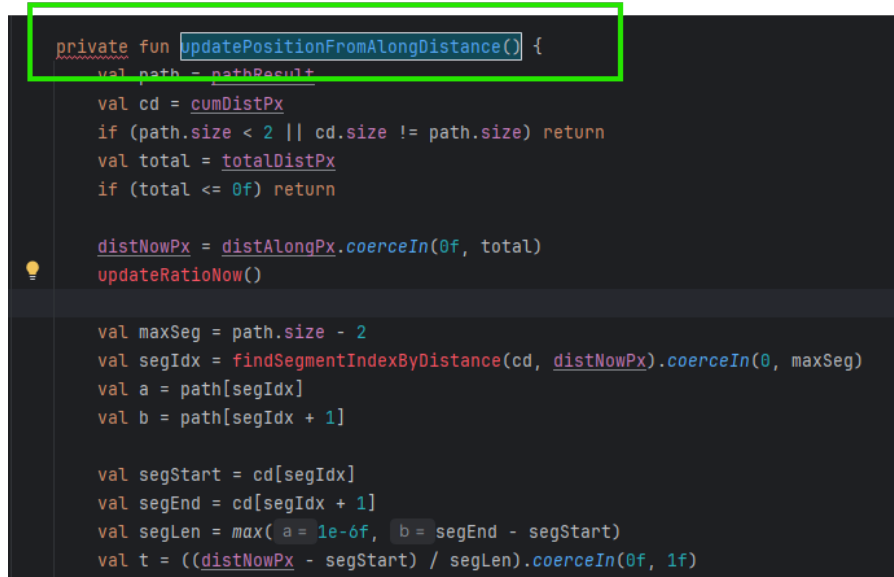
```

override fun onSensorChanged(e: SensorEvent?) {
    // Si on vient de sortir d'un lock, on applique les pas accumulés d'un
    if (pendingStepsWhileLocked > 0) {
        applyPendingSteps()
    }

    sentStepCount++
    val stepPx = stepLenPxNav()
    distAlongPx = min(a = totalDistPx, b = distAlongPx + stepPx)
    udpSendLine(
        "CTRL,STEPINFO,STEPLENM,%.3f,CAD,%.0f"
        .format(Locale.US, ...args = stepLenNavM, cadenceSpmNow())
    )
    val nowMs = System.currentTimeMillis()
    udpSendLine("STEP_EVT,$sentStepCount,$nowMs")
    updatePositionFromAlongDistance()
    Log.i(
        TAG,
        msg = "STEP stepPx=%.2f stepLenM=%.3f T=%.0fms cad=%.0fspm distAlongPx"
        .format(
    
```

FIGURE 3.13 : Mise à jour de la position de l'utilisateur après détection d'un pas

À partir de cette distance cumulée, l'application met à jour la position de l'utilisateur sur le trajet calculé. La position estimée est projetée sur le chemin afin de garantir que le déplacement reste cohérent avec l'itinéraire prévu. Cette mise à jour continue permet de suivre l'avancement de l'utilisateur même en l'absence de système de positionnement absolu comme le GPS.



```
private fun updatePositionFromAlongDistance() {  
    val path = pathResult  
    val cd = cumDistPx  
    if (path.size < 2 || cd.size != path.size) return  
    val total = totalDistPx  
    if (total <= 0f) return  
  
    distNowPx = distAlongPx.coerceIn(0f, total)  
    updateRatioNow()  
  
    val maxSeg = path.size - 2  
    val segIdx = findSegmentIndexByDistance(cd, distNowPx).coerceIn(0, maxSeg)  
    val a = path[segIdx]  
    val b = path[segIdx + 1]  
  
    val segStart = cd[segIdx]  
    val segEnd = cd[segIdx + 1]  
    val segLen = max(a = 1e-6f, b = segEnd - segStart)  
    val t = ((distNowPx - segStart) / segLen).coerceIn(0f, 1f)
```

FIGURE 3.14 : Projection de la position estimée sur le chemin calculé

En parallèle, l'orientation du téléphone est exploitée afin d'assurer une cohérence entre la direction du déplacement réel et la direction du trajet. Cette information est utilisée pour adapter l'affichage de la navigation et préparer la gestion des changements de direction, notamment lors des virages.

Enfin, le système met à jour différents indicateurs de navigation, tels que la progression globale sur le trajet et la distance restante jusqu'à la destination. Ces informations sont utilisées à la fois pour l'affichage à l'utilisateur et pour la logique interne de la navigation, permettant ainsi un suivi fluide, continu et compréhensible tout au long du déplacement.

3.4 Gestion intelligente des virages

La gestion des virages constitue un élément clé de la navigation indoor proposée. En effet, lors d'un changement de direction, les erreurs liées aux capteurs inertiels peuvent rapidement s'accumuler et dégrader la précision du suivi. Afin de garantir une navigation fluide et cohérente, une logique spécifique a été mise en place pour détecter les virages, contrôler leur exécution et sécuriser la transition entre les différents segments du trajet.

3.4.1 Détection des virages sur le trajet

Avant même que l'utilisateur n'atteigne un virage, l'application identifie les changements de direction présents sur le trajet calculé. Cette détection est réalisée à partir de l'analyse du chemin, en comparant l'orientation des segments successifs qui le composent.

Lorsque la variation d'orientation entre deux segments dépasse un certain seuil, un événement de virage est identifié. Cette information permet à l'application d'anticiper le changement de direction

et de préparer la logique de navigation associée. Ainsi, le système sait à quel moment un virage doit être effectué et peut adapter son comportement en conséquence.

La détection des virages joue un rôle fondamental, car elle permet de distinguer les phases de déplacement rectiligne des phases de changement de direction. Cette distinction est essentielle pour éviter des erreurs de positionnement lors des virages, où les capteurs sont généralement plus sensibles au bruit.

```
// ===== TURN EVENTS =====
private fun computeTurnEventsByDistance(path: List<PointF>, cd: FloatArray): List<TurnEvent> {
    if (path.size < 3) return emptyList()

    fun segAngle(i: Int): Float {
        val a = path[i]
        val b = path[i + 1]
        val dx = (b.x - a.x)
        val dy = (b.y - a.y)
        return normalizeAngle(
            a = Math.toDegrees(atan2(y = dx.toDouble(), x = (-dy).toDouble())).toFloat()
        )
    }

    val out = mutableListOf<TurnEvent>()
    for (i in 0 until path.size - 2) {
        val a1 = segAngle(i)
        val a2 = segAngle(i + 1)
        val d = normalizeAngle(a = a2 - a1)
        val len1 = hypotf(dx, dy)
        val len2 = hypotf(dx, dy)
        if (len1 > 18f && len2 > 18f && abs(x = d) >= 30f) {
            val dir = if (d > 0) "RIGHT" else "LEFT"
            val distAt = cd[i + 1]
            out.add(TurnEvent(atDistPx = distAt, dir, newHeadingAbs = a2))
        }
    }
}
```

FIGURE 3.15 : Détection des changements de direction le long du trajet

```
if (len1 > 18f && len2 > 18f && abs(x = d) >= 30f) {
    val dir = if (d > 0) "RIGHT" else "LEFT"
    val distAt = cd[i + 1]
    out.add(TurnEvent(atDistPx = distAt, dir, newHeadingAbs = a2))
}
```

FIGURE 3.16 : Implémentation du mécanisme de verrouillage de virage (Turn Lock)

3.4.2 Mécanisme de verrouillage de virage (Turn Lock)

Une fois un virage détecté, l'application active un mécanisme spécifique appelé Turn Lock. L'objectif de ce mécanisme est de contrôler strictement la phase de changement de direction afin d'éviter que la progression de l'utilisateur ne soit mise à jour de manière incorrecte.

Lorsqu'un virage est en cours, la progression linéaire sur le trajet est temporairement verrouillée. Pendant cette phase, les pas détectés ne sont plus utilisés pour avancer sur le chemin tant que le virage n'est pas correctement validé. La validation du virage repose principalement sur l'analyse de l'orientation du téléphone, qui permet de vérifier que l'utilisateur a effectivement changé de direction conformément au trajet prévu.

Ce mécanisme permet d'éviter les erreurs fréquentes liées aux mouvements parasites, aux

hésitations ou aux rotations incomplètes. Une fois le virage validé, la navigation reprend normalement et la progression sur le trajet est de nouveau autorisée.

```
private fun startTurnLock(dirRight: Boolean) {
    turnLockActive = true
    turnLockStartMs = System.currentTimeMillis()
    turnDirSign = if (dirRight) +1 else -1

    turnGyroZlp = 0f
    turnGyroHoldMs = 0L
    turnGyroAccumRad = 0f
    turnGyroAccumRadSigned = 0f
    turnGyroAngleDeg = 0f

    turnStartTsNs = 0L
    turnLastTsNs = 0L

    turnGyroPhase = 0 // 0: waiting opposite lobe, 1: waiting
    turnGyroStableCount = 0

    turnVibeDone = false
}
```

FIGURE 3.17 : Activation et initialisation du mécanisme de Turn Lock

```
private fun updateTurnLockByGyro(): Boolean {
    // 1) securite timeout (evite blocage infini)
    val nowMs = System.currentTimeMillis()
    if (turnLockStartMs != 0L && nowMs - turnLockStartMs > lockTimeoutMs) {
        // si on est aligné, on valide, sinon on relâche (fail-safe) pour ne pas rester bloqué
        val yawRel = getYawFiltered()
        val err = abs(angleErrorDeg(turnLockTargetRel, yawRel))
        if (err <= turnYawAcceptDeg) {
            validateTurn()
            return true
        } else {
            Log.w(TAG, "TURN LOCK TIMEOUT -> release (err=%.1f)".format(Locale.US, err))
            turnLockActive = false
            pendingStepsWhileLocked = 0
            turnLockSuppressed = true
            val supPx = (2f * stepLenPxNav()).coerceAtLeast(minimumValue = 20f)
            turnLockSuppressUntilDistPx = distAlongPx + supPx
            Log.w(
                TAG,
            )
        }
    }
}
```

FIGURE 3.18 : Gestion et contrôle du verrouillage pendant la phase de virage

```

private fun validateTurn() {
    turnGyroAccumRad = 0f
    turnGyroAccumRadSigned = 0f

    turnStartTsNs = 0L
    turnLastTsNs = 0L
    turnVibeDone = false
    lockOkCount = 0
    lockGyroOkCount = 0
    turnSawLargeErr = false
    lockMaxYawErrDeg = 0f
    turnLockStartMs = 0L
    turnLockActive = false
    nextTurnIdx++

    if (pendingStepsWhileLocked > 0) {
        applyPendingSteps()
        udpSendLine("STEP,$sentStepCount")
        draw()
    }
}

```

FIGURE 3.19 : Validation du virage et reprise de la progression

3.4.3 Stratégies de sécurité et cas particuliers

Afin de rendre le système plus robuste, plusieurs stratégies de sécurité ont été mises en place pour gérer les cas particuliers pouvant survenir lors des virages. Ces situations incluent notamment les virages incomplets, les rotations excessives ou l'absence de validation dans un temps raisonnable.

Dans ces cas, des mécanismes de temporisation et de réinitialisation sont utilisés pour éviter que l'application ne reste bloquée indéfiniment dans un état de virage. Le système peut ainsi sortir automatiquement du mode Turn Lock si les conditions attendues ne sont pas satisfaites, garantissant une continuité de la navigation.

Ces stratégies permettent d'assurer un compromis entre précision et robustesse, en maintenant une navigation stable même lorsque le comportement de l'utilisateur ou les données des capteurs ne sont pas idéales.

```

private fun updateTurnLockByGyro(): Boolean {
    // 1) securite timeout (evite blocage infini)
    val nowMs = System.currentTimeMillis()
    if (turnLockStartMs != 0L && nowMs - turnLockStartMs > lockTimeoutMs) {
        // si on est aligne, on valide; sinon on relache (fail-safe) pour ne pas rester bloqué
        val yawRel = getYawFiltered()
        val err = abs(angleErrorDeg(turnLockTargetRel, yawRel))
        if (err < turnYawAcceptDeg) {
            validateTurn()
            return true
        } else {
            Log.w(TAG, msg = "TURN LOCK TIMEOUT -> release (err=%.1f)".format(Locale.US, err))
            turnLockActive = false
            pendingStepsWhileLocked = 0
            turnLockSuppressed = true
            val supPx = (2f * stepLenPxNav()).coerceAtLeast(minimumValue = 20f)
            turnLockSuppressUntilDistPx = distAlongPx + supPx
            Log.w(
                TAG,
                msg = "TURN RESET (timeout). Suppress relock until dist=%.1fpx (+%.1fpx)"
                    .format(Locale.US, turnLockSuppressUntilDistPx, supPx)
            )
        }
    }
    return false
}

```

FIGURE 3.20 : Implémentation des mécanismes de sécurité et gestion des cas particuliers

3.5 Interaction utilisateur et retour visuel

L'interaction utilisateur constitue un élément essentiel de la solution proposée, car elle encadre toutes les étapes de la navigation, depuis la préparation initiale jusqu'à l'arrivée à destination. La première action demandée à l'utilisateur lors de l'utilisation de l'application est l'étalonnage. Cette étape permet d'adapter le système à ses caractéristiques de marche, notamment en calibrant la longueur de pas et les paramètres nécessaires à l'estimation du déplacement. Cet ajustement initial est indispensable pour garantir une meilleure précision tout au long de la navigation.

Une fois l'étalonnage effectué, l'utilisateur peut sélectionner le point de départ et la destination directement sur le plan intérieur. Cette interaction visuelle permet de définir le trajet de manière claire et adaptée à l'environnement du bâtiment. Après validation, la navigation est lancée et l'interface évolue pour afficher les informations utiles au guidage.

Pendant le déplacement, l'application fournit un retour visuel continu permettant de suivre la progression sur le trajet. La position estimée est affichée en temps réel sur le plan intérieur, accompagnée du chemin calculé et des indications de direction. L'interface met également en évidence les changements de direction afin d'aider l'utilisateur à anticiper les virages.

L'application intègre également des fonctionnalités de pause et de reprise de la navigation. L'utilisateur peut interrompre temporairement le suivi lorsqu'il le souhaite, ce qui suspend la mise à jour de la progression. La reprise permet de relancer la navigation à partir de la dernière position estimée, garantissant ainsi une utilisation flexible et adaptée aux situations réelles.

Ainsi, l'interface joue un rôle central en facilitant l'interaction, en rendant les informations compréhensibles et en assurant une expérience de navigation fluide et maîtrisée.

3.6 Communication avec MATLAB et outils d'analyse

Afin de faciliter l'analyse, la validation et le débogage du système de navigation indoor, une communication entre l'application Android et l'environnement MATLAB a été mise en place. Cette communication constitue un outil de support au développement et n'est pas indispensable au fonctionnement autonome de l'application.

L'application envoie vers MATLAB différentes données liées à la navigation, telles que les informations issues des capteurs, la progression de l'utilisateur ou encore certains paramètres de navigation. Ces échanges sont réalisés via le protocole UDP, choisi pour sa légèreté et son adaptation aux contraintes du temps réel.

Grâce à cette communication, il est possible de visualiser en temps réel l'évolution des données de navigation et d'analyser le comportement du système dans différentes situations. MATLAB permet

ainsi d’observer l’impact des déplacements, des virages et des changements d’orientation, facilitant l’identification d’éventuelles erreurs ou incohérences dans la logique de navigation.

Cette approche a joué un rôle important dans l’amélioration progressive de la solution. Elle a permis d’ajuster les paramètres, de valider les choix algorithmiques et de renforcer la robustesse globale du système. Il est important de souligner que, même en l’absence de cette communication externe, l’application reste pleinement fonctionnelle et capable d’assurer la navigation indoor de manière autonome.

Conclusion

Ce chapitre a permis de présenter en détail la conception et la logique de fonctionnement de la solution de navigation indoor proposée. Il a mis en évidence l’architecture globale de l’application, l’enchaînement des traitements, ainsi que les mécanismes assurant un suivi fiable de l’utilisateur et une gestion intelligente des virages. Ces choix de conception constituent une base solide pour garantir le bon fonctionnement du système. Le chapitre suivant sera consacré à l’évaluation de la solution à travers des tests, des résultats expérimentaux et des retours d’expérience.

RÉALISATION

Plan

1	Présentation de l'application finale	47
2	Mise en œuvre expérimentale et conditions de test	50
3	Résultats expérimentaux obtenus	51
4	Analyse et discussion des résultats	52
5	Retours d'expérience des utilisateurs	53
6	Perspectives et améliorations possibles	53

Introduction

Ce chapitre est consacré à la mise en œuvre pratique et à l'évaluation expérimentale de la solution développée. Après la conception et l'explication détaillée de l'architecture dans le chapitre précédent, l'objectif est ici de présenter l'application finale, les conditions de test mises en place ainsi que les résultats obtenus. Cette analyse permet d'évaluer les performances réelles du système et d'identifier ses points forts ainsi que ses limites.

4.1 Présentation de l'application finale

La version finale de l'application met en œuvre l'ensemble des fonctionnalités développées au cours de ce projet, depuis la phase d'étalonnage jusqu'au suivi complet de la navigation en temps réel. Afin de mieux illustrer son fonctionnement, les différentes étapes sont présentées à travers les captures d'écran suivantes.

La première étape lors de l'utilisation de l'application est l'étalonnage. Cette phase est indispensable car elle permet d'adapter la navigation aux caractéristiques de marche propres à chaque utilisateur. Une fenêtre explicative s'affiche afin de guider l'utilisateur et de préciser que la précision du système dépend de cette calibration préalable. L'objectif est d'obtenir une solution personnalisée, ajustée à la longueur de pas et au rythme de marche de la personne.

L'étalonnage est réalisé sur une distance fixe de 5 mètres, entre la porte de SUP Galilée et le premier poteau rencontré dans le couloir. Lorsque l'utilisateur est prêt, il clique sur «Commencer», l'application démarre l'enregistrement. Elle commence alors à compter le nombre de pas effectués ainsi que les éventuelles perturbations détectées, par exemple des mouvements brusques ou irréguliers. Une fois arrivé au point repère, l'utilisateur appuie sur «Arrêter». L'application calcule automatiquement la longueur moyenne du pas, la cadence de marche ainsi que la qualité du déplacement (marche normale, rapide ou mode balade). Ces informations permettent d'adapter le suivi de la navigation aux caractéristiques réelles de l'utilisateur.

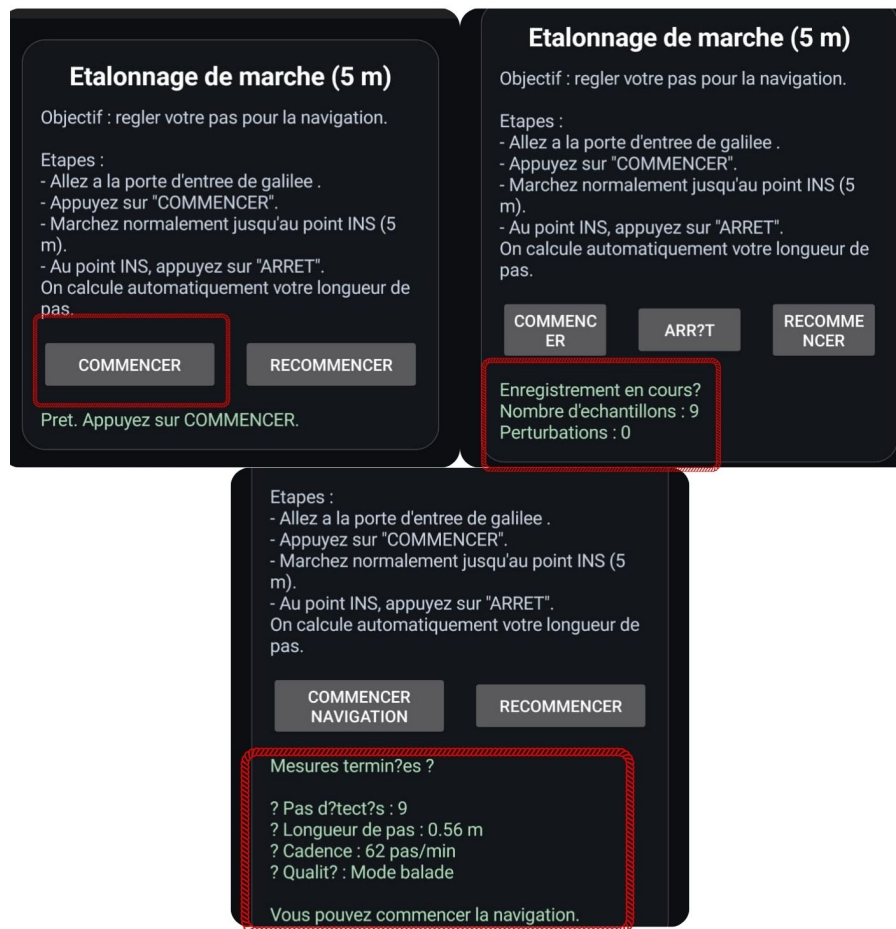


FIGURE 4.1 : Phase d'étalonnage personnalisé de la marche (5 m)

Après la phase d'étalonnage, l'utilisateur peut passer à la navigation. Comme le montre l'image suivante, la procédure est simple et intuitive. L'utilisateur commence par choisir son point de départ directement sur le plan intérieur, puis valide sa sélection. Il choisit ensuite la destination et confirme à nouveau. Une fois ces deux points définis, l'application calcule automatiquement le chemin le plus court à suivre.

Pendant le déplacement, la position est mise à jour en temps réel en fonction des pas détectés et de l'orientation du téléphone. Le trajet apparaît clairement sur le plan et l'utilisateur peut suivre sa progression de manière continue. En cas d'arrêt temporaire ou d'obstacle, il est possible de mettre la navigation en pause, puis de la reprendre sans perdre la progression déjà effectuée. Il est également possible de recommencer entièrement la navigation si nécessaire.

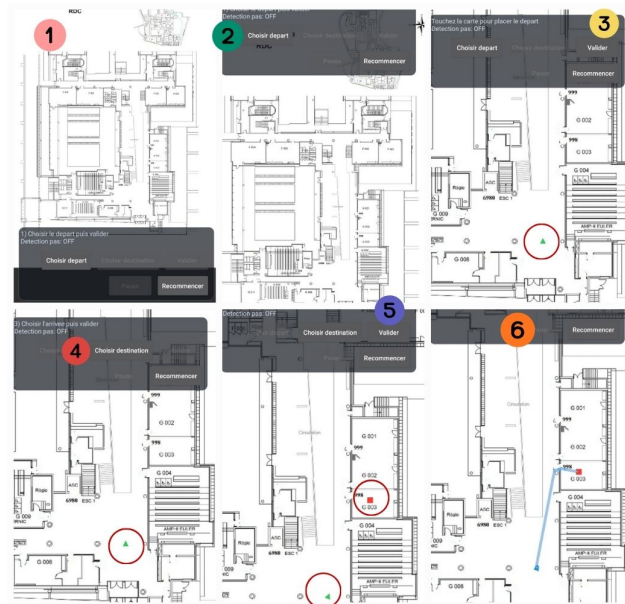


FIGURE 4.2 : Processus de définition du parcours sur le plan intérieur

La dernière image met en évidence les différents boutons disponibles pendant la navigation. Les fonctions « Pause », «Reprendre» et «Recommencer» offrent une utilisation flexible et adaptée aux situations réelles. Cette organisation permet à l'utilisateur de garder le contrôle total sur la navigation tout en bénéficiant d'un guidage continu et fiable.



FIGURE 4.3 : Fonctionnalités interactives de contrôle du parcours

Ainsi, l'application finale propose une solution complète, personnalisée et adaptée aux contraintes de la navigation indoor, tout en restant simple d'utilisation et ergonomique.

4.2 Mise en œuvre expérimentale et conditions de test

Après le développement de l'application, une phase expérimentale a été menée afin d'évaluer son fonctionnement en conditions réelles. L'objectif était de vérifier la fiabilité du suivi, la précision des estimations ainsi que le comportement global du système lors de déplacements naturels en environnement intérieur. Les tests réalisés ont permis d'observer les performances de la solution dans différentes situations et d'identifier ses limites éventuelles.

4.2.1 Environnement de test

Les expérimentations ont été réalisées à l'intérieur du bâtiment de SUP Galilée, dans un environnement réel comprenant couloirs, intersections et zones ouvertes. Le plan intérieur utilisé dans l'application correspond fidèlement à la configuration du rez-de-chaussée, ce qui a permis d'effectuer des tests cohérents et représentatifs.

Les essais ont été effectués à l'aide d'un smartphone Android équipé de capteurs inertiels intégrés (accéléromètre et gyroscope). Aucun système de positionnement externe (GPS, WiFi ou balises) n'a été utilisé, afin de tester exclusivement la performance de la navigation basée sur les capteurs embarqués.

Les conditions de test étaient réalistes : marche naturelle, présence éventuelle d'obstacles, arrêts temporaires et changements de direction multiples. Cela a permis d'évaluer la robustesse du système face aux variations normales du comportement humain.

4.2.2 Scénarios de test

Plusieurs scénarios ont été définis afin d'analyser le comportement du système dans différentes situations de navigation.

Le premier scénario consistait à suivre un trajet rectiligne sans changement de direction majeur, afin d'évaluer la précision de la détection des pas et l'estimation de la distance parcourue.

Le deuxième scénario intégrait plusieurs virages, permettant de tester la gestion intelligente des changements de direction ainsi que le mécanisme de verrouillage des virages. Ce cas était particulièrement important pour vérifier la cohérence du suivi lors des transitions entre segments du trajet.

Un troisième scénario incluait des pauses et des reprises de navigation afin de tester la stabilité du système et sa capacité à maintenir une estimation cohérente après une interruption.

Enfin, des tests complémentaires ont été réalisés avec des rythmes de marche différents (marche lente, normale et rapide) pour analyser l'impact de la cadence sur la précision du suivi.

Ces différents scénarios ont permis d'obtenir une vision globale des performances du système et de préparer l'analyse détaillée des résultats présentée dans la section suivante.

4.3 Résultats expérimentaux obtenus

Cette section présente les performances du système de navigation indoor développé, évaluées à travers plusieurs scénarios expérimentaux en environnement réel.

4.3.1 Environnement et protocole de test

Les expérimentations ont été réalisées dans les couloirs du bâtiment SUP Galilée. Les parcours incluait des segments rectilignes, des virages à 90 degrés ainsi que des trajets multi-segments.

Plusieurs utilisateurs ont participé aux tests afin d'évaluer l'adaptabilité du système à différents profils de marche (vitesse normale, marche rapide, mode balade).

Chaque test comportait :

- Une phase d'étalonnage sur une distance contrôlée de 5 mètres.
- Une phase de navigation entre un point de départ et un point d'arrivée.
- Une comparaison entre la distance réelle et la distance estimée.

4.3.2 Performance de la détection des pas

Les résultats montrent que l'algorithme de détection des pas, basé sur un filtrage passe-bande (0.6–3 Hz) et une validation des pics selon des contraintes physiques (amplitude et intervalle), permet une reconnaissance fiable des pas.

Le taux de faux positifs reste faible, même en présence de mouvements parasites ou de variations de vitesse.

4.3.3 Précision de l'estimation de distance

Après étalonnage personnalisé, la longueur moyenne du pas est calculée comme :

$$L_{pas} = \frac{D}{N}$$

où D représente la distance connue (5 m) et N le nombre de pas détectés.

L'erreur relative sur la distance estimée a été considérablement réduite grâce à la calibration. Les écarts observés restent limités et principalement liés aux variations naturelles de la marche.

4.3.4 Stabilité de l'orientation

L'estimation du cap (yaw) repose sur la fusion des capteurs inertiels. L'application d'un lissage léger et d'une correction des discontinuités ($359^\circ \rightarrow 0^\circ$) permet d'améliorer la stabilité directionnelle.

Lors des tests comportant plusieurs virages, le système a maintenu une cohérence satisfaisante entre l'orientation estimée et la trajectoire réelle.

4.3.5 Robustesse globale du système

Dans l'ensemble des scénarios testés, le système a démontré :

- Une détection stable des pas.
- Une estimation cohérente de la distance parcourue.
- Une gestion correcte des changements de direction.
- Une navigation fluide en temps réel.

Ces résultats confirment la faisabilité d'un système de navigation indoor basé uniquement sur des capteurs inertiels embarqués.

4.4 Analyse et discussion des résultats

Les résultats expérimentaux obtenus confirment la cohérence de l'approche adoptée pour la navigation indoor basée sur les capteurs inertiels du smartphone. La combinaison du Pedestrian Dead Reckoning (PDR), de la fusion de capteurs et de l'algorithme A* permet d'obtenir une solution fonctionnelle et adaptée à un environnement réel.

L'analyse des performances montre que la phase d'étalonnage joue un rôle déterminant dans la réduction de l'erreur systématique sur la distance parcourue. En personnalisant la longueur moyenne du pas et en estimant dynamiquement la cadence, le système s'adapte au profil biomécanique de l'utilisateur, ce qui améliore significativement la précision globale.

Concernant l'orientation, la fusion des capteurs et la stabilisation appliquée au yaw permettent de maintenir une trajectoire cohérente, notamment lors des changements de direction. Cependant, certaines dérives peuvent apparaître sur des parcours longs, principalement en raison des limitations intrinsèques des capteurs inertiels, telles que la dérive du gyroscope ou les perturbations magnétiques.

Les tests ont également mis en évidence l'importance des conditions réelles d'utilisation. Les variations de vitesse, les arrêts temporaires ou les mouvements brusques peuvent influencer temporairement l'estimation. Néanmoins, les mécanismes de filtrage et de validation des pas contribuent à limiter l'impact de ces perturbations.

Dans l'ensemble, les performances observées démontrent que la solution proposée constitue une base solide pour un système de navigation indoor autonome. Les limites identifiées ouvrent néanmoins la voie à des améliorations futures, notamment en matière de fusion de capteurs avancée et de correction adaptative des erreurs.

4.5 Retours d'expérience des utilisateurs

Afin d'évaluer l'ergonomie et l'utilisabilité de l'application, plusieurs utilisateurs ont testé le système dans des conditions réelles de navigation. Les retours obtenus permettent d'analyser non seulement les performances techniques, mais également l'expérience globale d'utilisation.

De manière générale, les utilisateurs ont apprécié la simplicité de l'interface et la clarté des étapes, notamment la phase d'étalonnage et la sélection des points de départ et d'arrivée. La visualisation du trajet sur le plan et la mise à jour en temps réel de la position ont été perçues comme intuitives et faciles à suivre.

La fonction de mise en pause et de reprise de la navigation a également été jugée utile, en particulier lors de situations imprévues (arrêt momentané, obstacle, interaction extérieure). Cette flexibilité renforce l'aspect pratique de l'application en conditions réelles.

Certains utilisateurs ont toutefois remarqué de légères déviations lors de longues trajectoires ou après plusieurs virages successifs. Ces observations confirment les limites identifiées dans l'analyse technique et soulignent l'importance d'améliorations futures.

Globalement, les retours sont positifs et montrent que le système est fonctionnel, compréhensible et exploitable par un utilisateur non expert. Ces retours valident la pertinence de l'approche adoptée et encouragent le développement de versions plus avancées intégrant des mécanismes d'auto-correction et d'apprentissage adaptatif.

4.6 Perspectives et améliorations possibles

Bien que les résultats obtenus soient encourageants, plusieurs axes d'amélioration peuvent être envisagés afin d'augmenter la précision et la robustesse de la solution proposée. En effet, la navigation indoor basée uniquement sur les capteurs inertiels présente certaines limites liées notamment à la dérive de l'orientation et à l'accumulation progressive des erreurs.

Une première perspective d'amélioration concerne l'intégration de techniques de fusion de capteurs plus avancées. L'utilisation combinée de filtres adaptés, comme des filtres complémentaires ou des filtres de Kalman, permettrait d'améliorer la stabilité de l'estimation de l'orientation et de réduire l'impact du bruit des capteurs. Cela pourrait contribuer à limiter la dérive observée lors de trajets comportant plusieurs changements de direction.

Une autre amélioration possible serait l'intégration de sources d'information complémentaires, telles que des balises Bluetooth Low Energy (BLE) ou des signaux WiFi. Ces technologies pourraient servir de points de correction ponctuels afin de recalibrer la position estimée et d'éviter l'accumulation d'erreurs sur de longues distances.

Du point de vue algorithmique, des méthodes d'apprentissage automatique pourraient également être explorées pour améliorer la détection des pas et l'analyse du comportement de marche. Une adaptation dynamique des paramètres en fonction du profil de l'utilisateur pourrait rendre la navigation encore plus personnalisée et précise.

Par ailleurs, des améliorations ergonomiques peuvent être envisagées au niveau de l'interface utilisateur, notamment l'ajout d'indications vocales, d'alertes vibratoires ou d'un guidage plus interactif. Ces éléments pourraient améliorer l'accessibilité de l'application, en particulier pour des utilisateurs ayant des besoins spécifiques.

Enfin, une optimisation énergétique constitue également une perspective intéressante. La gestion intelligente de l'activation des capteurs permettrait de réduire la consommation de batterie, rendant la solution plus adaptée à une utilisation prolongée.

Ainsi, bien que l'application développée constitue une base fonctionnelle et cohérente pour la navigation indoor, plusieurs pistes d'évolution peuvent être envisagées afin d'augmenter sa précision, sa robustesse et son adaptabilité dans des environnements plus complexes.

Conclusion

Ce chapitre a permis de présenter concrètement l'application développée et de valider son fonctionnement à travers des tests en conditions réelles. Les résultats obtenus confirment la faisabilité d'une navigation indoor basée uniquement sur les capteurs du smartphone. Malgré certaines limites liées aux capteurs inertiels, le système reste stable, cohérent et exploitable. Ce travail démontre ainsi la pertinence de l'approche proposée et constitue une base solide pour de futures améliorations.

Conclusion générale

Ce projet avait pour objectif la conception et le développement d'une application de navigation indoor basée exclusivement sur l'exploitation des capteurs inertiels intégrés au smartphone. Face aux limites des technologies de positionnement traditionnelles en environnement intérieur, telles que le GPS, ce travail a permis d'explorer une approche autonome fondée sur le Pedestrian Dead Reckoning (PDR) et la fusion de capteurs.

Au cours de ce projet, nous avons conçu une architecture complète intégrant plusieurs modules complémentaires : la détection et la validation des pas, l'estimation dynamique de l'orientation (yaw), le calcul du chemin optimal à l'aide de l'algorithme A*, ainsi que des mécanismes de sécurisation tels que le verrouillage de virage (Turn Lock). L'ensemble de ces composants a été développé, testé et intégré dans une application fonctionnelle.

Les expérimentations réalisées en environnement réel ont permis de valider la faisabilité de la solution proposée. Les résultats obtenus montrent une détection fiable des pas, une estimation cohérente de la distance après étalonnage personnalisé, ainsi qu'une gestion satisfaisante des changements de direction. Malgré certaines limites inhérentes aux capteurs inertiels, notamment la dérive du gyroscope et la sensibilité aux perturbations magnétiques, le système a démontré une robustesse globale acceptable dans des conditions d'utilisation réelles.

Au-delà de l'aspect technique, ce projet a également permis d'approfondir des notions essentielles telles que le traitement du signal, la fusion de capteurs, la planification de trajectoire et la conception d'interfaces interactives. Il a constitué une expérience enrichissante, tant sur le plan académique que pratique.

Enfin, les perspectives d'amélioration identifiées notamment l'intégration de techniques d'intelligence artificielle, l'amélioration de la fusion de capteurs et l'extension à des environnements multi-bâtiments ouvrent la voie à une évolution vers un système de navigation indoor plus intelligent, plus adaptatif et plus universel.

Ce travail constitue ainsi une base solide pour le développement futur de solutions de navigation intérieure autonomes et évolutives.

